

ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA

SCUOLA DI INGEGNERIA E ARCHITETTURA
DIPARTIMENTO DI INFORMATICA – SCIENZA E INGEGNERIA
CORSO DI LAUREA IN
INGEGNERIA INFORMATICA

TESI DI LAUREA
in
TECNOLOGIE WEB T

Tecniche Avanzate
di Sviluppo Chatbot
basate su LLM e RAG

CANDIDATO:
CESARE TOMASI

RELATORE:
Chiar.mo Prof. Ing. Paolo Bellavista

Anno Accademico 2024/2025
Sessione Estiva

Sommario

Introduzione	4
Capitolo I - Overview Teorica	6
Definizione di LLM (large language model)	6
Definizione di RAG (retrieval-augmented generation) e motivi per cui si è ricorsi ad approcci basati su di esso	6
Architettura di base e flusso di lavoro tipico	7
Embedding: scelta del tipo della rappresentazione e dall'algoritmo di misura della somiglianza	9
Embedding: tecniche di misura della somiglianza	10
Limiti del RAG tradizionale	11
Tipologie avanzate di RAG	11
Capitolo II - Tecnologie utilizzate	13
LangChain: introduzione	13
LangChain: interfacce dei modelli	14
LangChain: invocazione di tool	14
LangChain: caching dei risultati	15
LangGraph: gestione della memoria	16
LangGraph: agenti	18
LlamaIndex: introduzione	19
LlamaIndex: overview tecnica	20
LlamaIndex: connettori di dati	21
LlamaIndex: indici di dati	22
LlamaIndex: modelli di embedding	23
LlamaIndex: query engine	24
LlamaIndex: chat engine	24
LlamaIndex: workflow	24
LlamaIndex: valutazione e benchmarking	25
LLaMA 2: descrizione, personalizzazioni e deployment	27
Streamlit: introduzione e descrizione del modello di funzionamento	29
Capitolo III - descrizione e realizzazione del progetto “Assistente UniBo”	32
Descrizione del caso d'uso	32
Considerazioni iniziali sull'ambiente d'esecuzione	32
LangChain e LlamaIndex: componenti software utilizzati	34
Architettura logica del progetto	34
Diagramma della struttura del progetto (Directory Tree)	35
Analisi e commento del sorgente app.py	36
Analisi e commento del sorgente rag_init.py	38
Analisi e commento del sorgente web_app.py	40
Analisi e commento del sorgente evaluation.py - introduzione	41

Analisi e commento del sorgente evaluation.py - funzione load_documents()	43
Analisi e commento del sorgente evaluation.py - funzione generate_test_set()	44
Analisi e commento del sorgente evaluation.py - funzione create_fallback_questions()	46
Analisi e commento del sorgente evaluation.py - funzione parse_qa()	47
Analisi e commento del sorgente evaluation.py - classe LocalEvaluator	49
Analisi e commento del sorgente evaluation.py - funzione evaluate_rag()	50
Analisi e commento del sorgente evaluation.py - funzione visualize_performance()	52
Analisi e commento del sorgente evaluation.py - main()	53
Considerazioni sulle performance generali del chatbot	55
Assistente UniBo: galleria di screenshot	56
Conclusioni	58
Bibliografia	59

Sommario delle figure

- Figura 1 - Schema generale della pipeline RAG - [Pagina 7]
- Figura 2 - Schema di dettaglio della pipeline RAG - [Pagina 9]
- Figura 3 - Tipi di rappresentazioni vettoriali - [Pagina 9]
- Figura 4 - Rappresentazione della misura della somiglianza - [Pagina 10]
- Figura 5 - Logo della libreria LangChain - [Pagina 13]
- Figura 6 - Schema di un tool moltiplicatore - [Pagina 15]
- Figura 7 - Interazione LLM con e senza con cache - [Pagina 16]
- Figura 8 - Tipi di memorie di un LLM - [Pagina 17]
- Figura 9 - Spettro delle architetture degli agenti - [Pagina 19]
- Figura 10 - Logo della libreria LlamaIndex - [Pagina 20]
- Figura 11 - Schema connettore dati per encoding - [Pagina 21]
- Figura 12 - Schema VectorStoreIndex - [Pagina 22]
- Figura 13 - Schema Summary Index - [Pagina 22]
- Figura 14 - Schema Index Tree - [Pagina 22]
- Figura 15 - Schema processo di embedding - [Pagina 23]
- Figura 16 - Schema query engine - [Pagina 24]
- Figura 17 - Esempio di workflow - [Pagina 25]
- Figura 18 - Schema di Response Evaluation - [Pagina 27]
- Figura 19 - Logo della famiglia di modelli LLaMA - [Pagina 28]
- Figura 20 - Schema di utilizzo di Ollama - [Pagina 28]
- Figura 21 - Logo della libreria Streamlit - [Pagina 29]
- Figura 22 - Schema architetturale dell'utilizzo di Streamlit - [Pagina 31]
- Figura 23 - Specifiche della macchina remota utilizzata per il progetto - [Pagina 33]
- Figura 24 - Schema della pipeline di processamento dei documenti e indicizzazione degli embedding - [Pagina 34]
- Figura 25 - Schema dell'architettura logica del chatbot realizzato - [Pagina 35]
- Figura 26 - Albero dei componenti del progetto - [Pagina 35]
- Figura 27 - Esempio di tupla di performance.csv - [Pagina 54]
- Figura 28 - Schermata 1 di interazione con il chatbot - [Pagina 56]
- Figura 29 - Schermata 2 di interazione con il chatbot - [Pagina 56]
- Figura 30 - Schermata 3 di interazione con il chatbot - [Pagina 57]

Introduzione

Seppur originariamente coniata negli anni '50 del secolo scorso, è negli ultimi anni che stiamo assistendo sempre di più alla popolarizzazione dell'espressione intelligenza artificiale, la quale fa riferimento ad un campo di studi che tratta la creazione di sistemi informatici che possano replicare il modo di ragionare e risolvere i problemi della mente umana.

Ci si potrebbe chiedere per quale ragione si sia aspettato così tanto: i recenti sviluppi di tali tecnologie sono principalmente dovuti alla sempre maggiore disponibilità e qualità dei dati che sono accessibili liberamente dai ricercatori.

A tale proposito, è interessante definire il concetto di apprendimento automatico (o machine learning): esso è una branca dell'intelligenza artificiale che si occupa della definizione di algoritmi e tecniche che permettano ai computer di apprendere autonomamente dai dati forniti dal programmatore, senza che esso debba definire esplicitamente quali operazioni debba eseguire, cosa che permette al sistema risultante di adattarsi dinamicamente alle condizioni descritte dalle fonti d'informazione utilizzate, oltre che di risolvere problemi difficilmente affrontabili diversamente per via della loro complessità [1].

Esempi di compiti di questo genere sono: il riconoscimento e la sintesi vocale; la robotica; la visione artificiale; la risoluzione di giochi e rompicapi; l'elaborazione del linguaggio umano.

Quest'ultimo campo è quello che verrà approfondito in questa tesi: facendo uso delle tecnologie LLM e RAG (successivamente definite), verrà realizzato un chatbot (un vero e proprio assistente virtuale) in grado di rispondere correttamente a domande riguardanti i regolamenti dell'Università degli Studi di Bologna (usando come fonte i numerosi documenti presenti presso la sezione "Normateneo" consultabili liberamente dal sito istituzionale).

Nello specifico, il chatbot sfrutta dunque: un LLM della famiglia LLaMAntino per la creazione del testo (fondamentale per un supporto adeguato alla lingua italiana); le librerie open source LangChain e LlamaIndex per la creazione della pipeline RAG (composta da caricamento dei documenti, indicizzazione e ricerca semantica degli stessi, seguita infine della generazione delle risposte); la libreria Streamlit per la realizzazione del front-end web dell'applicativo, necessario per l'interazione con l'utente finale; la piattaforma Ollama per il deployment locale del modello.

L'impiego di componenti software open source, unito alla possibilità di selezionare corpus di testi diversi a seconda del caso d'uso preso in esame, rende la soluzione scelta particolarmente modulare e riutilizzabile.

La tesi avrà infine la seguente struttura: nel primo capitolo verranno introdotti i concetti teorici necessari per comprendere il funzionamento ad alto livello dell'intero progetto; nel secondo si descriveranno le tecnologie utilizzate e nel terzo verrà illustrata l'effettiva implementazione e una metodologia di valutazione delle performance.

Capitolo I - Overview Teorica

Definizione di LLM (large language model)

Gli LLM, cioè i modelli linguistici di grandi dimensioni (definiti tali per via del numero considerevole di parametri presenti all'interno delle reti neurali di riferimento), sono una categoria di modelli di machine learning capaci di comprendere ed analizzare il linguaggio naturale [2].

Sono inoltre considerati modelli di fondazione (**foundation models**): sono cioè caratterizzati dal fatto di venire addestrati su un'ampia gamma di dati non strutturati, cosa che li rende particolarmente adattabili ai più diversi domini di applicazioni [3], tra i quali possiamo citare: la rielaborazione e il riassunto di testi complessi; la traduzione automatica e accurata in lingue straniere; la generazione di codice eseguibile; la creazione di chatbot [2].

In linea di massima, gli LLM funzionano generando testo basandosi sul contesto dato e sulle parole prodotte in precedenza, oltre che concezioni di natura probabilistica apprese durante la fase di addestramento della rete [3].

Definizione di RAG (retrieval-augmented generation) e motivi per cui si è ricorsi ad approcci basati su di esso

Per quanto un LLM sia capace di conservare quantità ingenti d'informazioni sottoforma di parametri interni alla rete stessa, tale approccio risulta poco affidabile (rischio non trascurabile di diffusione di informazioni false, imprecise, o superficiali), oltre che non così facilmente aggiornabile nel tempo [4].

Rende inoltre difficile anche riconoscere le fonti effettive di tali risultati, cosa che ne limita la verificabilità.

Dunque, in tutte le situazioni in cui tali limiti non sono sostenibili è necessario ricorrere ad un'alternativa: l'idea di base dietro al RAG, è dunque quella di avere una soluzione ibrida, che integri ai dati già parametrizzati all'interno del LLM con quelli provenienti da una fonte esterna d'informazioni, come un documento o una base dati, rendendo l'intera soluzione più flessibile e migliorando sensibilmente la qualità delle risposte, senza la stretta necessità di ricorrere ad addestramento specifico del modello stesso.

Questa è infatti uno degli aspetti principali che distingue il RAG (più adatto alle situazioni in cui avere accesso ad informazioni aggiornate in tempo reale è fondamentale) dal Fine Tuning: quest'ultimo approccio richiede infatti il riaddestramento del LLM a partire da dati specifici di un determinato dominio di applicazione (con l'obiettivo di adattarlo meglio ad esso), operazione che però chiaramente ha un costo in termini di tempo non indifferente [5].

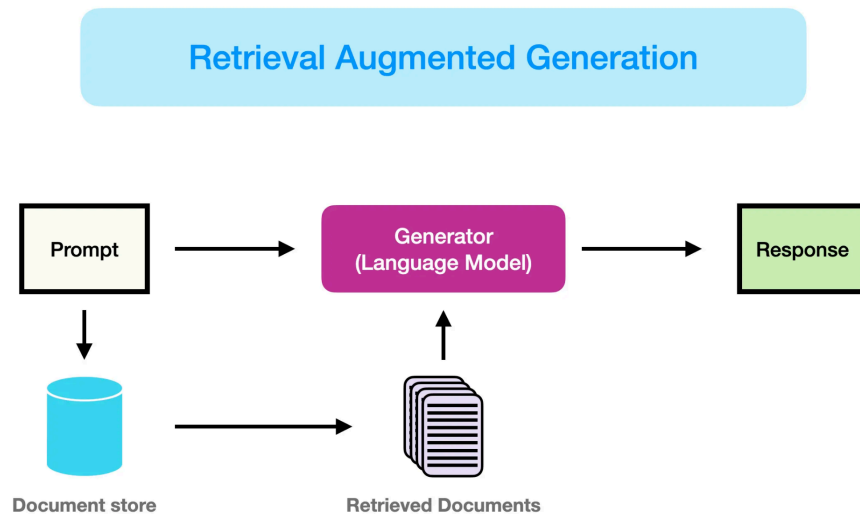


Figura 1 - Schema generale della pipeline RAG

Esempi significativi di utilizzi di questa tecnologia possono essere: l'analisi di documentazione complessa, come di natura medica, legale o finanziaria; il miglioramento dei suggerimenti e dei risultati di ricerca in piattaforme di e-commerce di qualsivoglia natura; la creazione di chatbot intelligenti che sfruttino abilmente fonti di informazioni esterne, utili negli ambiti dell'istruzione, dell'assistenza personale e del supporto clienti.

Architettura di base e flusso di lavoro tipico

In primo luogo, sarà necessario caricare in locale (fase di **document loading**) i file (solitamente di testo), che verranno utilizzati come fonti aggiuntive di informazioni per il modello, in modo da poter portare avanti un'operazione di parsing su di essi e da poterli formattare correttamente.

Successivamente, saranno soggetti ad un'operazione di separazione del testo in segmenti (detti chunk) di dimensioni limitate (fase di **document splitting**), che risulteranno chiaramente più facilmente gestibili dall'intero sistema.

Quest'ultimo aspetto non è poi così semplice come potrebbe sembrare: bisognerà infatti fare in modo che tale divisione avvenga tenendo conto della semantica dei documenti originali, che dovrà dunque essere preservata [5].

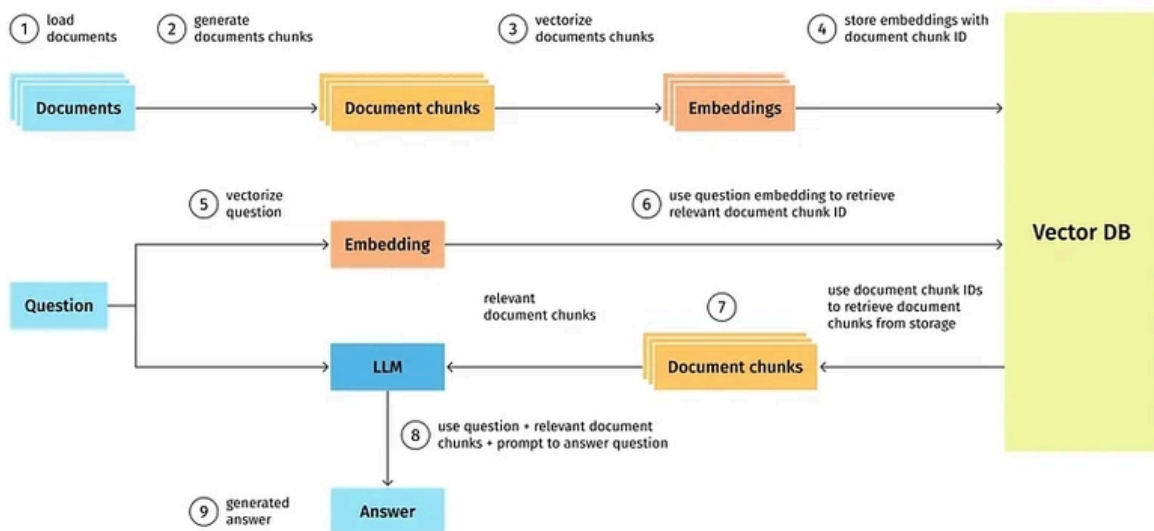
Infine, per permettere al modello di accedere in modo efficiente al contenuto delle sorgenti esterne utilizzate, è necessario effettuare un'operazione di codifica di tali informazioni, che verranno incapsulate in forma strutturata all'interno di vettori numerici multidimensionali (detti anche **embedding**), che ne catturano il significato indipendentemente dalla rappresentazione [4].

Questi a loro volta saranno conservati all'interno di database vettoriali (detti **vector store**), che permetteranno l'effettuazione di query che tengano conto delle relazioni semantiche presenti tra le varie fonti d'informazione, anche se formulate in modi completamente diversi (difficilmente fattibile con dati non strutturati) [5].

Esempi di database di questo tipo sono: FAISS (sviluppato da Meta, conosciuto per le sue performance elevate e dunque adatto a sistemi che devono gestire una mole considerevole di dati) e Chroma (open source e caratterizzato da una forte scalabilità e flessibilità, oltre che dalla possibilità di hosting sia locale che remoto) [5].

Una volta completato tale procedimento preparatorio, l'intero processo si sviluppa dunque in tre passi: in primo luogo avremo una fase di recupero (**retrieval**), in cui verranno selezionati gli embedding rilevanti alla query effettuata dall'utente (quelli che risulteranno più simili ad essa); in secondo luogo, avremo una fase di arricchimento (augmentation) del prompt dato con le informazioni raccolte nella fase precedente (che diventeranno quindi una fonte di contesto aggiuntivo); infine ci sarà invece una fase di generazione (**generation**), in cui LLM genererà una risposta adeguata alla richiesta effettuata [5].

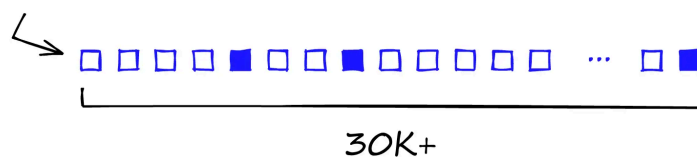
Inoltre, la query digitata dall'utente potrà essere potenzialmente arricchita in modi diversi per rendere più fruttuosa ed efficace la fase di retrieval, aggiungendo: sinonimi o termini legati a quelli già presenti (**synonym expansion**) e vocaboli facenti riferimento a concetti collegati a quelli già espressi (**conceptual expansion**) [6].



Infine, un'ulteriore metodologia per migliorare i risultati della fase di retrieval corrisponde all'effettivo filtraggio dei risultati, tenendo conto dei: metadati, (**metadata-based filtering**), come data o tipo di documento; del contenuto effettivo (**content-based filtering**), definendo quindi delle threshold minime di rilevanza basate su criteri semantici o sulla presenza consistente di parole chiave [6].

Embedding: scelta del tipo della rappresentazione e dall'algoritmo di misura della somiglianza

sparse

$$[0, 0, 0, 1, 0, \dots, 0]$$


dense

$[0.2, 0.7, 0.1, 0.8, 0.1, \dots 0.9]$

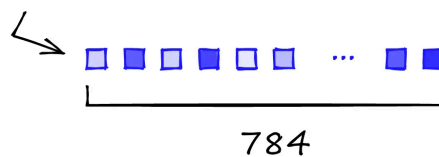


Figura 3 - Tipi di rappresentazioni vettoriali

Esistono due principali tipologie di rappresentazione vettoriale di un documento in linguaggio naturale, oltre che agli ovvi approcci ibridi (spesso tenenti conto sia della

semantica che delle keywords del testo considerato [6]) che ne alternano le caratteristiche: quelle sparse (o **sparse representation**), caratterizzate dall'uso di vettori molto lunghi con tante dimensioni quante parole sono presenti nel vocabolario scelto; quelle dense (o **dense representation**), in cui invece sono impiegati vettori di dimensione minore (maggiore efficienza) e che tengono conto della semantica del testo originale (testi dal significato analogo saranno rappresentati da vettori simili tra loro), cosa che non era altrettanto vero per il caso precedente [7].

Solitamente tale risultato viene reso possibile impiegando un modello (per esempio **Word2Vec**) capace di riconoscere le relazioni esistenti tra le parole del testo originale, oltre che la frequenza con cui esse si presentano [6].

Esempi invece di rappresentazioni sparse sono: **bag-of-words**, in cui si pone ad uno il valore delle celle del vettore corrispondenti a parole presenti nel testo e a 0 quelle non presenti; **TF-IDF** (acronimo di Term Frequency - Inverse Document Frequency), in cui invece vengono associati pesi alle parole in base al numero di occorrenze di esse presenti (dando un peso minore alle parole presenti in gran numero, essendo esse solitamente poco significative); **BM-25** (simile a TF-IDF, ma progettato per essere indipendente dalla lunghezza del documento originale, cosa che sennò porterebbe ad una maggiore rilevanza a priori, riferita alle query, di quelli più estesi rispetto a quelli più corti) [7].

Embedding: tecniche di misura della somiglianza

Va specificato inoltre che nella fase di retrieval, la query stessa, originariamente espressa in linguaggio naturale, viene a sua volta tradotta in una rappresentazione numerica vettoriale, cosa che risulta fondamentale per rendere possibile il confronto con gli embedding precedentemente creati.

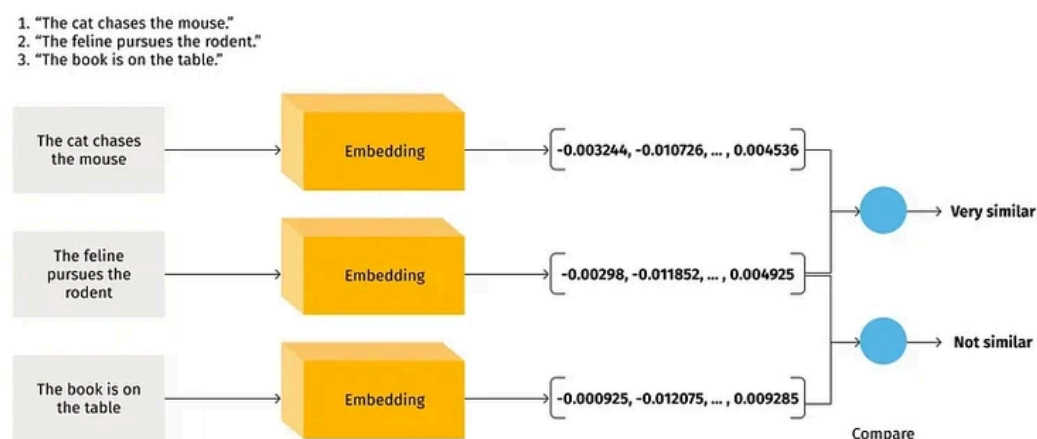


Figura 4 - Rappresentazione della misura della somiglianza

A tal proposito, esistono varie tecniche per effettuare tale operazione, tra le quali possiamo citare: il prodotto scalare, o **dot product** (vantaggioso perché veloce da calcolare, ma di uso limitato per via della sua dipendenza dal modulo dei vettori coinvolti, limitazione condivisa anche dal calcolo della distanza euclidea e di quello del prodotto interno massimo, o **MIPS**); la similarità del coseno, o **cosine similarity**, cioè la misura del coseno dell'angolo formato tra i due vettori (vantaggiosa per via della sua indipendenza dalla scala dei vettori, cosa che l'ha resa molto popolare in questo ambito) [8][9].

Limiti del RAG tradizionale

Seppur funzionali e sufficienti in molti contesti, gli approcci descritti in precedenza risultano essere soggetti ad una serie di problematiche ed inefficienze [6]: difficoltà nel gestire conversazioni lunghe ed a mantenere in memoria il contesto delle stesse; gestione impropria delle query specifiche del dominio d'interesse, cosa che porta al recupero di informazioni inutili o poco rilevanti; rischio di allucinazioni, seppur in misura minore a ciò che accade in assenza di RAG.

Tipologie avanzate di RAG

L'approccio RAG non è monolitico, ma è invece possibile personalizzarlo e adattarlo liberamente, in modo da poter meglio soddisfare i propri specifici requisiti.

Ad oggi sono sei le tipologie più popolari [5]:

- RAG nativo (Native RAG), caratterizzato dalla forte integrazione presente tra le componenti di recupero e generazione delle informazioni (in contrapposizione agli approcci più classici, solitamente fortemente dipendenti da API esterne), cosa che lo rende più veloce e meno soggetto a latenza;
- RAG per Recupero e Riclassificazione (Retrieve and Rerank RAG), che si distingue per via dell'impiego di un ulteriore modello che si occupa di riclassificare gli embedding selezionati nella fase di retrieval, in modo da dare più priorità e importanza a quelli maggiormente rilevanti alla query effettuata;
- RAG multimodale (Multimodal RAG), il quale impiega tipologie di dati differenti oltre al semplice testo (immagini, video, audio, eccetera) sia in fase di recupero che di generazione delle informazioni;

- RAG a grafo (Graph RAG), nel quale le informazioni trattate sono rappresentate attraverso strutture a grafo, in modo da meglio mostrare le relazioni e gerarchie presenti tra esse (molto utile nella fase di retrieval);
- RAG ibrido (Hybrid RAG), che fa uso di diversi algoritmi di retrieval contemporaneamente, cosa che lo rende più preciso quando si ha a che fare con un numero considerevole di dati, soprattutto se molto vari tra loro;
- RAG ad Agenti (Agentic RAG), che può prevedere uno strato decisionale (router) che analizza la query di input e sceglie la metodologia di recupero e generazione delle informazioni più adeguata per gestirla, o che è invece basato su una rete di agenti specializzati, ognuno con un compito preciso [10].

In entrambi questi ultimi casi, avremo una forte flessibilità ed adattabilità, oltre che un'eccellente scalabilità.

Capitolo II - Tecnologie utilizzate

LangChain: introduzione

LangChain è un framework progettato per semplificare la creazione di applicativi che sfruttino gli LLM attraverso la concatenazione di componenti distinti [11].

Si tratta fondamentalmente di una serie di moduli utilizzabili indipendentemente l'uno dall'altro, ma che insieme formano un package Python [12].



Figura 5 - Logo della libreria LangChain

Le funzionalità più importanti che esso offre sono le seguenti: standardizzazione delle interfacce dei modelli linguistici e dei componenti associati, che se usate nativamente potrebbero essere invece radicalmente differenti tra loro a seconda del caso specifico (generalmente contenute nel pacchetto **langchain-core**, con le integrazioni più pesanti che sono state separate in pacchetti a parte, come **langchain-openai** o **langchain-anthropic**, e quelle di terze parti che sono invece contenute in **langchain-community**); supporto avanzato all'orchestrazione (orchestration) dei moduli interessati, gestendone cioè la comunicazione e coordinazione (si potrà eventualmente ricorrere al pacchetto **langgraph** per i progetti più complessi); capacità di provvedere all'osservabilità e alla valutazione dei risultati, in modo da poter scegliere consapevolmente quale tipo di soluzione (per esempio modelli o tipologie di prompt) utilizzare nella propria applicazione (si potrà eventualmente ricorrere al pacchetto **langsmith** per i progetti più complessi).

Infine, il pacchetto **langchain** contiene le componenti software necessarie per la costruzione di agenti e per la gestione delle tecniche di retrieval delle informazioni.

LangChain: interfacce dei modelli

I modelli supportati nativamente da LangChain (come ad esempio AWS Bedrock, DeepSeek o Google Gemini) implementano l'interfaccia standard **BaseChatModel**.

I metodi più rilevanti che essa offre sono i seguenti:

- **invoke()**, il quale costituisce la via di comunicazione primaria con il modello (accetta una lista di messaggi, formattati secondo lo standard di OpenAI o quello di LangChain, come input e ne restituisce uno come output;
- **stream()**, che permette il reperimento del testo di output in tempo reale alla generazione dello stesso;
- **batch()**, che permette l'invio in gruppo di più richieste, in modo da rendere più efficiente l'elaborazione delle stesse;
- **with_structured_output()**, un wrapper per il metodo `invoke()` per gestire il supporto agli output strutturati (come ad esempio i file JSON);
- **bind_tools()**, per legare il modello ad un eventuale strumento (o **tool**), cioè un qualche modulo o sistema software, come un database, un API o anche solo banalmente una funzione locale demarcata dal decoratore **@tool** [11].

Per quanto riguarda il supporto alla multimodalità (cioè alla capacità di ricevere come input o come output dei file multimediali, come foto o musica), essa è garantita solo per un numero limitato di modelli.

Infine, l'interfaccia fornisce una serie di parametri standard per scegliere e modificare il comportamento del LLM sottostante, i seguenti sono i più rilevanti:

- **model** (permette di indicare il nome specifico del modello che si vuole utilizzare);
- **temperature** (valore numerico che rappresenta il grado di randomicità/creatività delle risposte del modello);
- **timeout** (numero massimo di secondi di attesa di una risposta da parte del modello);
- **max_tokens** (controlla la lunghezza massima della risposta del LLM);
- **max_retries** (numero massimo di tentativi falliti in sequenza).

LangChain: invocazione di tool

A tal proposito è importante che le firme delle funzioni invocate come strumenti siano il più possibile descrittive e ben fatte (vari fattori risultano particolarmente importanti per ottenere

risultati ottimali: il nome della funzione, il tipo e il numero dei parametri, ma anche la docstring della funzione): per interagire correttamente con il tool, LLM dovrà infatti rispettare il formato dell'input richiesto da quest'ultimo, invece di rispondere in linguaggio naturale, come farebbe con un utente umano.

Se la risposta fornita dal LLM dovesse comprendere il risultato di una o più chiamate ad uno strumento, l'attributo **tool_calls** di essa conterrà una lista di oggetti **ToolCall**, ognuno dei quali specificante: il nome della funzione; il numero e il nome degli argomenti; l'id e il tipo della chiamata.

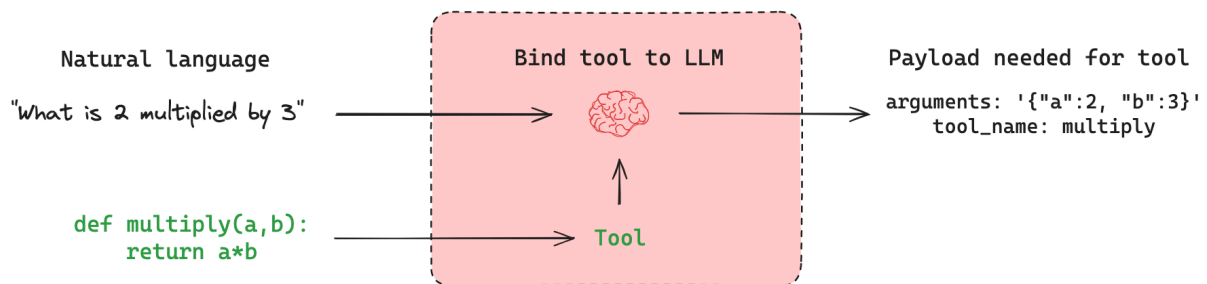


Figura 6 - Schema di un tool moltiplicatore

In caso il LLM dovesse invece generare una stringa di input malformata, portando dunque il tool a fallire e a non restituire un input valido, di tale chiamata verrà tenuto conto come oggetto **InvalidToolCall** (che conterrà il nome della funzione; il numero e il nome degli argomenti; l'id e un messaggio d'errore) di una lista associata all'attributo **invalid_tool_calls**.

Di default, il modello decide liberamente quando invocare i tool associati ad esso basandosi su quella che è stata la richiesta dell'utente, ma è possibile forzarne la chiamata specificando il nome dello strumento voluto all'interno del parametro **tool_choice** del metodo `bind_tools()` [11].

LangChain: caching dei risultati

Le API fornite dagli LLM tendono a non fornire risposte immediate, sia che i modelli siano in esecuzione in locale sia che lo siano in remoto.

Per tale ragione, può risultare giustificabile l'idea di salvare gli output dati dal modello in una memoria di cache, in modo da poterli recuperare facilmente e velocemente da essa, senza necessità di interrogare il LLM, cosa che risulterà fondamentale (per ragioni economiche) soprattutto nei casi in cui il modello è remoto.

D'altro canto però, è improbabile che la stessa domanda venga formulata allo stesso modo due volte: per tale ragione bisognerà quantomeno salvare nella cache le risposte generate in base alla semantica dell'input dato, cosa che risulterà utile anche per velocizzare la gestione di tutte quelle domande che sono poste di frequenza (FAQ).

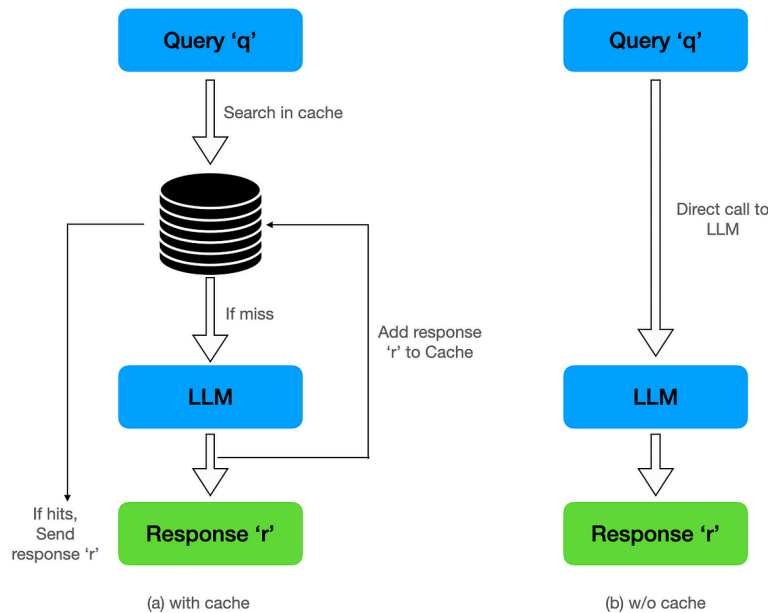


Figura 7 - Interazione LLM con e senza con cache

Nel pratico, per abilitare l'utilizzo della cache, basterà importare dal pacchetto **langchain_core** una classe a scelta tra **InMemoryCache** (che salva i risultati delle chiamate al modello in memoria), **SQLiteCache** (che salva i risultati delle chiamate al modello in un database esterno) o un'altra implementazione analoga, e richiamare il metodo **set_llm_cache()**, dandogli come argomento un oggetto delle classi appena citate.

LangGraph: gestione della memoria

Gli applicativi che fanno uso di LLM necessitano di uno stato che permetta loro di tenere conto dei dettagli rilevanti delle interazioni passate.

LangGraph supporta due tipi di memoria: quella a **breve termine**, che ha come tempo di vita la durata della sessione e che non è condivisa tra thread diversi, e quella a **lungo termine**, che è persistente e che può essere recuperata da ogni thread ed in qualsiasi momento.

La memoria a breve termine viene solitamente utilizzata per permettere al LLM di tenere traccia di una conversazione tenuta con un utente umano.

In assenza di essa, il modello genererebbe un output che non tiene in alcun modo conto dei messaggi precedenti.

Nel pratico, tali informazioni vengono salvate all'interno della struttura dati centralizzata **state** (tale stato sarà gestito da uno **StateGraph**, che si potrà ottenere istanziando un costruttore con il metodo **StateGraph()** e invocando su di esso il metodo **compile()**, aggiornata automaticamente ad ogni interazione (messaggio dell'utente, risposta dal modello, eccetera) grazie all'uso dei checkpoint, che possono salvarne il contenuto in memoria volatile (**InMemorySaver**) o centrale e che andranno inseriti come argomento del metodo **compile()** in fase di creazione dello **StateGraph**.

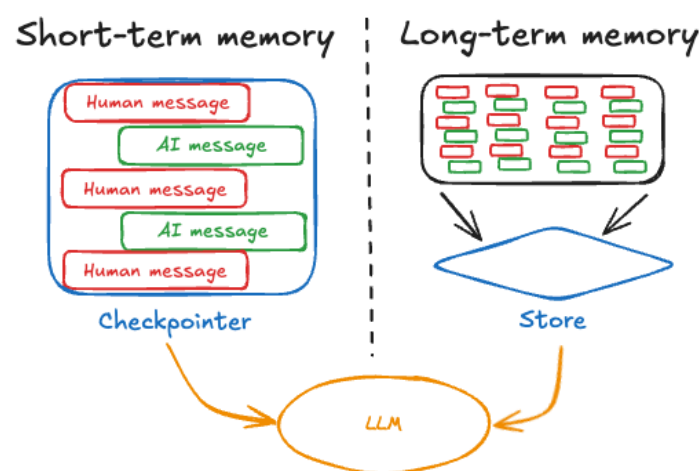


Figura 8 - Tipi di memorie di un LLM

In alternativa, è anche possibile usare un oggetto della classe **ConversationBufferMemory** (implementante l'interfaccia **BaseChatMemory**) per avere un mezzo più basico (ma senza supporto a funzionalità di alto livello, come la chiamata a tool esterni) per la gestione dello stato di una conversazione (memoria a breve termine).

A tal proposito, si useranno i metodi **add_user_message()** e **add_ai_message()**, per arricchire il contenuto dell'oggetto **ConversationBufferMemory** e il metodo **load_memory_variables()** per ottenerne il contenuto (specificandone la chiave associata).

Chiaramente la lunghezza massima dell'input accettata dai diversi LLM (la finestra di contesto o **context window**) non è infinita e potrà dunque essere necessario eliminare le componenti meno rilevanti dello stato.

Per quanto riguarda la memoria a lungo termine, essa è conservata sotto forma di file JSON all'interno di uno **store di memoria**, che può essere salvato in memoria locale (**InMemoryStore**), in memoria permanente o anche in un database esterno.

Gli elementi memorizzati all'interno di esso saranno organizzati in una gerarchia a due livelli: in primo luogo, vi sarà uno spazio di nomi (spesso facente riferimento all'identificativo di un utente o di un'azienda coinvolta); in secondo luogo, una chiave specifica (che identifica l'elemento univocamente in quel particolare spazio di nomi).

Nel pratico, sarà possibile usare il metodo **put()** per aggiungere un elemento allo store desiderato, specificando quale namespace e chiave utilizzare, oltre che un riferimento all'elemento stesso.

Per ottenere una lista degli ultimi caricamenti in uno store (il più recente si troverà all'ultimo posto mentre quello più vecchio si troverà al primo), sarà inoltre sufficiente invocare su di esso il metodo **search()**, ottenendo come output una lista di oggetti **Item**, convertibili in strutture dato dizionario con il metodo **dict()**.

Essi presenteranno i seguenti parametri: **value**, cioè il contenuto dell'elemento memorizzato (a sua volta codificato come dizionario); **key** e **namespace** relativi all'elemento memorizzato; **created_at** e **updated_at**, cioè dei timestamp indicanti il momento della creazione e di ultimo aggiornamento della memoria [11].

LangGraph: agenti

Può capitare che un'applicazione basata su LLM debba eseguire una serie di operazioni specifiche direttamente prima o dopo una chiamata al modello stesso.

Invece di rendere tali flussi di controllo rigidi e statici, risulta interessante rendere la scelta di quale scegliere tra gli stessi fluida e basata sulla risposta generata dal modello: ed è infatti proprio questo che fanno gli agenti in LangChain e LangGraph (costruibili facilmente attraverso il metodo **create_react_agent()**).

Per portare avanti tale obiettivo, è chiaro che sfruttino diverse delle tecnologie esposte in precedenza: output strutturato; chiamate a tool esterni; memoria a breve e a lungo termine.

Sono diverse le possibili architetture che un agente può avere (e che sono offerte come architetture standard da LangGraph) e in linea di massima si distinguono tra loro principalmente per il grado di controllo decisionale concesso al LLM: nel caso dei **Router**, al modello viene chiesto di scegliere tra una serie finita di opzioni possibili, limitandone di molto l'autonomia; nel caso invece degli agenti totalmente autonomi (o Fully **Autonomous**),

in cui il modello viene richiamato varie volte all'interno di un loop while e ad ogni step l'agente determina quale tool utilizzare e con quale input.

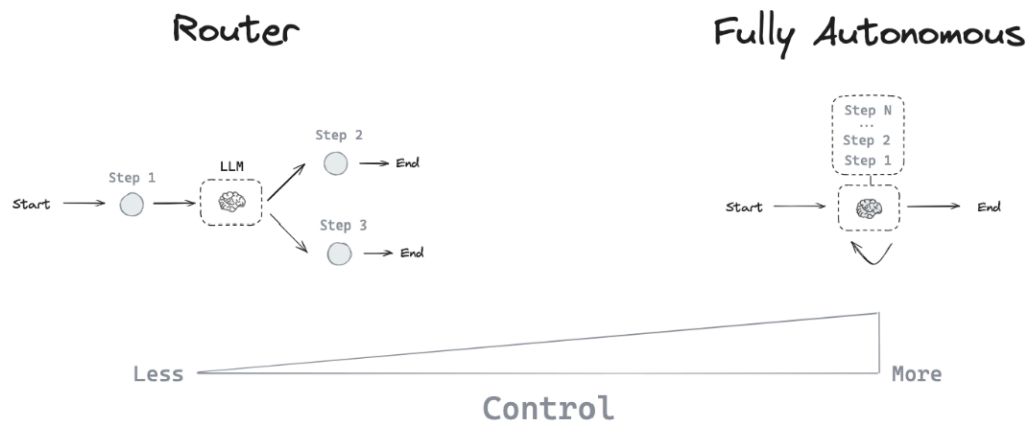


Figura 9 - Spettro delle architetture degli agenti

Successivamente l'output fornito dagli strumenti invocati viene ridato come input al modello e il ciclo continua fino a quando l'agente non risulta convinto di aver raccolto abbastanza informazioni per soddisfare la richiesta dell'utente [11].

LlamaIndex: introduzione

LlamaIndex è una libreria (utilizzabile via Python o TypeScript) che permette la costruzione di sistemi basati su RAG [13].

Garantisce un recupero dei dati veloce, preciso e affidabile a partire da diverse fonti esterne di informazioni e formati dati, basandosi principalmente su criteri semantici, cosa che lo rende un componente particolarmente adatto ai casi d'uso ricchi di testo.

Oltre a ciò, è caratterizzato da una forte minimalità, che rende possibile la creazione di applicativi in tempi brevi e con un numero limitato di linee di codice.

Un aspetto interessante è la sua capacità di integrarsi facilmente con altri framework esistenti, come ChatGPT, Flask, Docker e appunto LangChain [12].

Ci si potrebbe chiedere per quale ragione si dovrebbe voler ricorrere anche a LangChain, piuttosto che limitarsi al solo LlamaIndex se il proprio scopo è semplicemente quello di creare un applicativo che sfrutti l'approccio RAG.



Figure 10 - Logo della libreria LlamaIndex

Sono in realtà diverse le funzionalità aggiuntive offerte da LangChain [12]: supporto a fonti di dati multimodali, oltre al solo testo e immagini; controllo granulare di ogni fase del flusso di lavoro; mantenimento del contesto della conversazione (fondamentale per un qualsiasi chatbot); supporto a diverse tecniche di retrieval, come quella per parole chiave (non è dunque limitato alla sola ricerca semantica).

Quest'ultimo aspetto è molto importante, visto che permette in questo modo la gestione da parte del sistema di query molto più complesse rispetto al caso in cui esso non fosse presente.

LlamaIndex: overview tecnica

LlamaIndex fornisce una serie di strumenti fondamentali per la costruzione di un sistema basato su RAG: supporto nativo ai più popolari LLM e **modelli di embedding**, eseguiti sia in locale sia remotamente e dunque da interrogare tramite API (ad esempio i modelli della famiglia GPT di OpenAI); numerosi **connettori di dati**, per leggere ed estrarre informazioni da diverse sorgenti/formati dati (come API o documenti in formato PDF); **indici di dati**, contenenti gli encoding vettoriali delle informazioni gestite dall'applicativo da realizzare; componenti software per la gestione di query e conversazioni con l'utente (**query engine** e **chat engine**).

Oltre ad essi abbiamo anche una serie di moduli software altrettanto importanti, ma di natura più avanzata: **agenti**, intesi come sottosistemi asincroni che usano un LLM (guidato da un prompt specifico), una memoria di stato (la cui dimensione è personalizzabile invocando il metodo `ChatMemoryBuffer.from_defaults()` e specificando un limite di token) e una serie di strumenti (solitamente funzioni o classi definite ad hoc) per rispondere alle richieste di un utente reale; **workflow**, che rappresentano un mezzo asincrono e basato su eventi attraverso il quale è possibile controllare il flusso d'esecuzione di un'applicazione basata su LLM; strumenti sofisticati per la gestione dell'**osservabilità**, che rendono possibile, a scopo di debug o di ottimizzazione, osservare gli input e gli output di ogni componente software in

tempo reale o consultare lo storico delle operazioni di indicizzazione o di formulazione query e la **valutazione** del nostro applicativo [14].

LlamaIndex: connettori di dati

I connettori di dati sono implementati nel pratico come oggetti (da istanziare dunque dalla classe di riferimento prima dell'uso con un costruttore pubblico, i cui argomenti potranno essere utilizzati a scopi di configurazione e personalizzazione del comportamento, cosa che vale in realtà anche per gli indici di dati) e forniscono dei metodi per estrarre informazioni dalla sorgente desiderata.

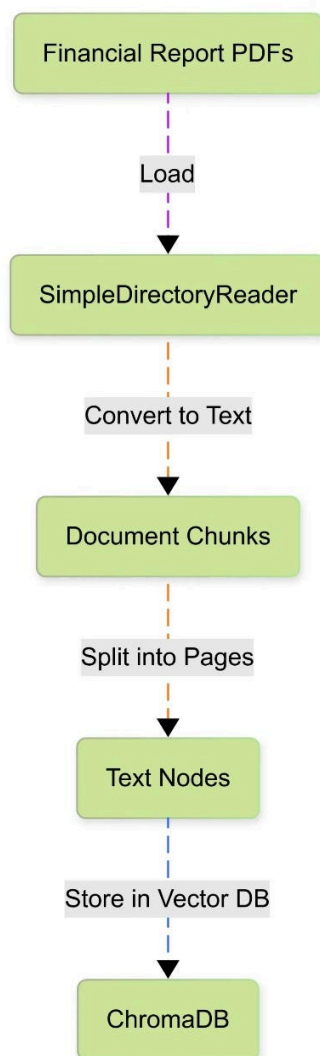


Figura 11 - Schema connettore dati per encoding

Ognuno di essi presenta parametri di inizializzazione e metodi leggermente diversi, ma il più semplice ed utilizzato è sicuramente il **SimpleDirectoryReader**, capace di fare il parsing dei tipi più comuni di file contenuti in una cartella disponibile localmente (il cui percorso verrà specificato attraverso il parametro **input_dir** del costruttore dell'oggetto).

Per limitarsi invece ad uno o più file specifici, si userà invece il parametro **input_files**, premunendosi di non porre il parametro **Recursive** a true, necessario invece per il caso precedente.

Il metodo che permette il caricamento sincrono del contenuto dei file letti è **load_data()**, mentre **aload_data()** è usato per quello asincrono (accompagnato dalla parola chiave **await**) [14].

LlamaIndex: indici di dati

Gli indici di dati sono particolari strutture dati che velocizzano il recupero d'informazioni da parte di un sistema RAG.

All'interno di essi le informazioni sono immagazzinate sottoforma di oggetti **Node**, che contengono porzioni dei documenti originali (rappresentati come oggetti **Document**) ottenuti attraverso i connettori di dati (che vengono spezzettati in seguito ad un processo di chunking), oltre che metadati che ne descrivono le relazioni con altri nodi.

Il **VectorStoreIndex** (all'interno del quale i nodi hanno una rappresentazione vettoriale) è il più utilizzato, ma non è l'unico: abbiamo per esempio anche il **Summary Index** (in cui i nodi sono posti come elementi di una catena sequenziale) e l'**Index Tree** (che sfrutta una struttura ad albero).

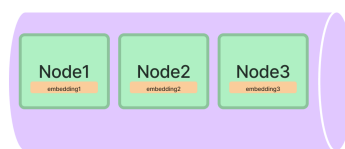


Figura 12 - Schema VectorStoreIndex

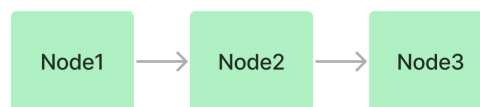


Figura 13 - Schema Summary Index

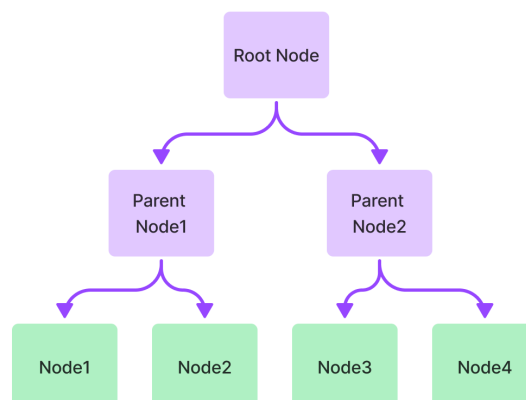


Figura 14 - Schema Index Tree

Prendendo come riferimento il `VectorStoreIndex`, il mondo più semplice per crearne uno è attraverso il metodo `from_documents()`, senza che sia necessario specificare alcun parametro di basso livello, come la `chunk_size` e i metadati, ma richiedendo invece tassativamente un riferimento al modello di embedding utilizzato.

Per maggiore controllo, risulta possibile definire una **IngestionPipeline** (personalizzabile in modo granulare attraverso i parametri di istanziazione) ed eventualmente gestendo la creazione dei nodi manualmente (dandoli poi come argomento al costruttore del `VectorStoreIndex`).

Infine, per conservare in memoria di massa una specifica istanza di un `VectorStoreIndex` (in modo da non doverla ricreare ogni volta a partire dai dati originali), si può ricorrere al metodo `storage_context.persist()`, che ci permetterà di specificare come uno **StorageContext**, cioè il percorso di un direttorio dove verrà salvato l'indice e da cui potrà successivamente essere recuperato attraverso il metodo `load_index_from_storage()` [14].

LlamaIndex: modelli di embedding

La conversione di informazioni non strutturate in embedding, necessaria per il funzionamento del `VectorStoreIndex`, è effettuata da modello di embedding, che agisce dunque sia in fase di creazione dell'indice che di inserimento della query (anch'essa dovrà essere disponibile in forma vettoriale per rendere possibile il paragone della stessa e dei vettori presenti all'interno dell'indice).

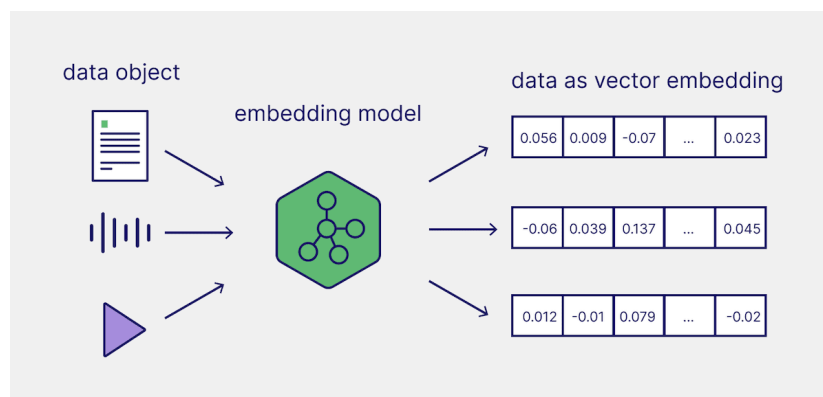


Figura 15 - Schema processo di embedding

Solitamente le soluzioni più affidabili ed efficaci a livello locale sono quelle offerte da **HuggingFace** (sul remoto invece OpenAI risulta preferibile): in tali situazioni si userà il costruttore `HuggingFaceEmbedding`, che avrà come argomenti il nome del modello considerato, la `embed_batch_size` (il numero di nodi o documenti che verranno codificati

contemporaneamente) e il tipo di device impiegato per portare avanti l'intero processo (CPU o GPU) [14].

LlamaIndex: query engine

Un query engine è un modulo software che rende possibile l'interrogazione tramite linguaggio naturale di una sorgente dati, solitamente un indice (nel caso specifico di LlamaIndex se ne può istanziare uno invocando il metodo **as_query_engine()**, specificando eventualmente quale LLM impiegare per la generazione delle risposte testuali come argomento, e si usa il metodo **query()**, con argomento un messaggio testuale, per ottenere un effettivo riscontro).

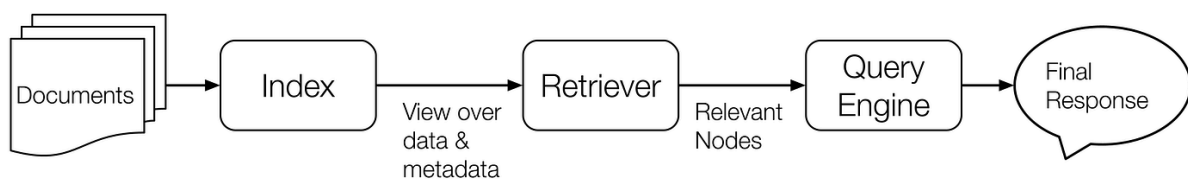


Figura 16 - Schema query engine

Solitamente essi risultano costruiti a partire da retriever, cioè moduli software che selezionano le informazioni più rilevanti ad una specifica query da una fonte d'informazioni. Anche in questo caso sono facilmente ottenibili da un indice attraverso il metodo **as_retriever()** [14].

LlamaIndex: chat engine

Un chat engine differisce dal query engine perché, a differenza di quest'ultimo, presenta uno stato ed è dunque in grado di sostenere una vera e propria conversazione, invece di essere solamente capace di rispondere ad una singola domanda non avendo memoria delle precedenti. Analogamente ai componenti precedenti, può essere istanziato a partire da indici preesistenti, in questo caso invocando il metodo **as_chat_engine()** si otterrà un'istanza dell'oggetto voluto, mentre si userà il metodo **chat()** dello stesso per aggiungere un ulteriore messaggio alla conversazione [14].

LlamaIndex: workflow

Il paradigma dei workflow prevede che l'applicazione venga divisa in passaggi (ognuno dei quali verrà segnalato nel decoratore **@step**) con un numero arbitrario di parametri input o di output, la cui esecuzione viene scatenata dal verificarsi di uno o più specifici eventi e che generano a loro volta uno o più di essi durante il proprio tempo di vita.

I vari step non verranno dunque eseguiti in modo sincrono e sequenziale tra di loro, ma in modo evidentemente asincrono.

L'organizzazione in passaggi ed eventi rende possibile organizzare il codice della logica di business del programma in modo che esso risulti più facile da leggere e manutenibile, cosa che risulta estremamente importante nel caso di applicazioni complesse e non lineari, rendendo possibile la divisione di problemi vasti e complicati in unità più semplici da gestire. Vale infine la pena notare che la maggior parte degli eventi impiegati in tali workflow saranno definiti dall'utente e saranno coerenti con quella che è la logica applicativa desiderata.

A quest'ultima regola fanno eccezione gli eventi **StartEvent** e **StopEvent**, che segnalano rispettivamente l'inizio ed il termine del flusso di lavoro [14].

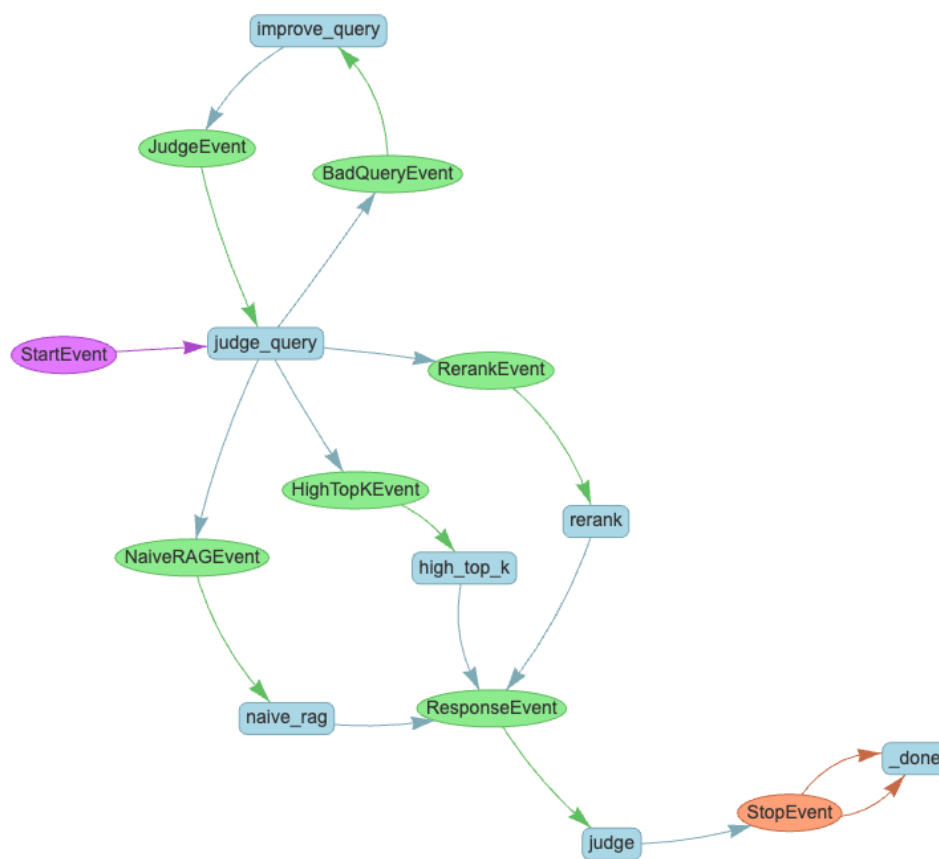


Figura 17 - Esempio di workflow

LlamaIndex: valutazione e benchmarking

Misurare e valutare la performance di un sistema basato su RAG è fondamentale per comprenderne dunque le potenzialità ed eventualmente per migliorare e mandarne avanti lo sviluppo.

Sono due i tipi di valutazione offerti da LlamaIndex: quella delle risposte generate (o **Response Evaluation**) e quella del recupero dei documenti rilevanti alla query (**Retrieval Evaluation**).

Nel primo caso, si paragonano i responsi generati dal LLM impiegato a quelli di uno ritenuto ampiamente affidabile (solitamente GPT-4) e se ne valuta:

- la **correttezza**, cioè la somiglianza sintattica e semantica tra la risposta data e quella di riferimento;
- la **vicinanza semantica**, cioè la vicinanza in termini di significato tra la risposta data e quella di riferimento;
- la **fedeltà**, intesa rispetto ai contenuti recuperati (è una misura del grado di allucinazioni generate);
- la rilevanza rispetto al **contesto generale**, alla **domanda effettuata** e alle **guidelines** fornite per la generazione della risposta.

Nel pratico tale operazione viene effettuata richiamando il metodo **evaluate()** (con parametri **query**, **contexts**, **response**) di un classe di tipo **Evaluator**, come **BaseEvaluator** o **FaithfulnessEvaluator**.

Inoltre, ogni chiamata del metodo **evaluate()** restituisce un oggetto **EvaluationResult**, che avrà i campi: **passing** (di tipo booleano), **score** (valore numerico intero) e **feedback** (riscontro di tipo testuale).

Per quanto riguarda il secondo invece, solitamente si considerano invece le seguenti operazioni: **generazione di un dataset** di coppie domande/risposte a partire da un corpus di testo non formattato (facilmente realizzare in LlamaIndex invocando la funzione **generate_question_context_pairs()**, specificando come parametri i nodi e LLM da utilizzare per l'operazione di sintesi, oltre il numero di domande per chunk di documento, che corrisponde a 512 parole); valutazione del recupero d'informazioni (o **retrieval evaluation**), attraverso la quale si compara l'opera di selezione (effettuata da retriever) di un set di domande dato rispetto a delle metriche standard, come la **precisione** o l'**hit-rate** (misura di quanto spesso uno specifico documento risulta rilevante a query effettuate e viene dunque recuperato).

Quest'ultima operazione viene resa possibile nel pratico dal metodo **evaluate** (che prenderà una query come argomento) di un **RetrieverEvaluator**.

Con un metodo della famiglia **from_metrics()** (come **from_metric_names()**), si potrà specificare quali metriche usare per la valutazione [14].

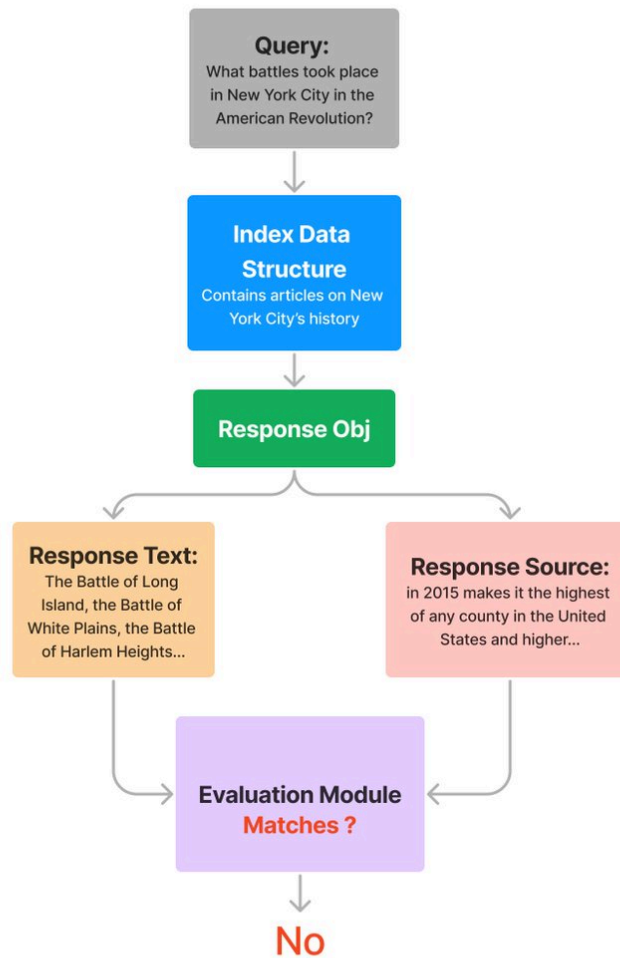


Figura 18 - Schema di Response Evaluation

LLaMA 2: descrizione, personalizzazioni e deployment

LLaMA 2 è un modello linguistico di grandi dimensioni rilasciato da Meta AI nel 2023. A differenza di altri progetti analoghi, sia il sorgente che i pesi utilizzati dalla rete sono stati resi completamente open source [15].

Disponibile in una serie di varianti, dai 7 ai 70 miliardi di parametri, in fase di sviluppo è stata data una particolare attenzione al miglioramento delle prestazioni di quelle più leggere, essendo esse anche quelle più accessibili all'utente comune.

A tal proposito, va specificato come tali prestazioni maggiorate siano rese possibili attraverso un processo di quantizzazione dei pesi impiegati dal modello: solitamente infatti essi sono rappresentati da numeri a virgola mobile a 16 o 32 bit, ma a scopo di riduzione del carico possono essere sostituiti da interi ad 8 bit, cosa che porterà ad una perdita di informazioni e precisione più o meno evidente a seconda del grado di quantizzazione impiegato (ad ognuno dei quali corrisponde una diversa versione del LLM) [16].

Inoltre, per tutti quei casi d'uso in cui risulta essere necessario un supporto migliorato alle lingue distinte dall'inglese (tra cui l'italiano), possono essere particolarmente interessanti i modelli della famiglia **LLaMAntino**, rilasciati dal Semantic Web and Personalization Research Lab, un avanguardistico laboratorio di ricerca del Dipartimento d'Informatica dell'Università di Bari [17]: ognuno di essi risulta basato su una versione di LLaMA opportunamente personalizzata (attraverso tecniche di fine tuning supervisionato) per meglio comprendere un idioma straniero di riferimento [18].



Figura 19 - Logo della famiglia di modelli LLaMA

Infine, ognuno di tali modelli può essere installato ed eseguito localmente con facilità ricorrendo alla piattaforma open source **Ollama** (Omni-Layer Learning Language Acquisition Model), una soluzione scalabile e flessibile (oltre che reperibile banalmente clonando il repository ufficiale su GitHub) che si occuperà di gestire in automatico anche tutte le dipendenze necessarie [19].

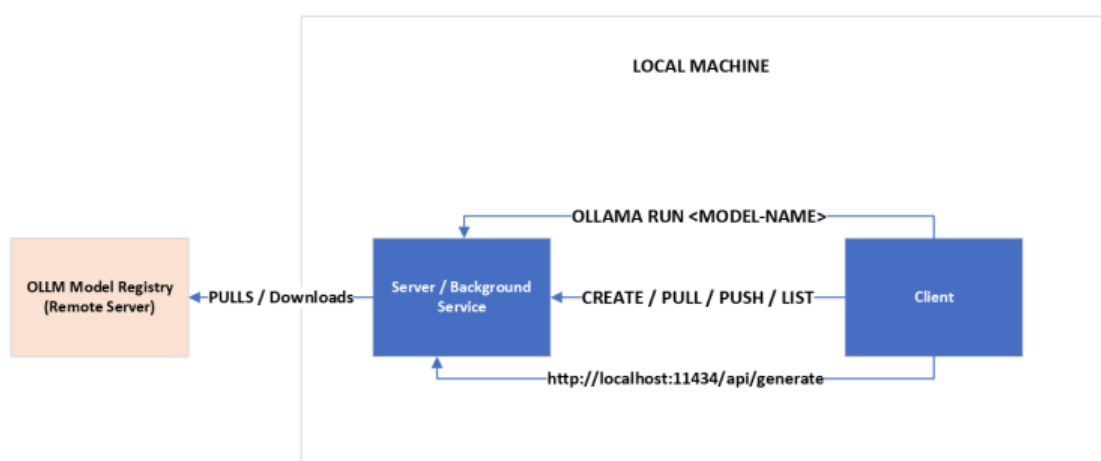


Figura 20 - Schema di utilizzo di Ollama

Nello specifico: per importare un nuovo modello si potrà ricorrere al comando **ollama pull** [20] o in alternativa definire un Modelfile, in cui verranno specificati una serie di parametri

necessari, tra cui il tipo e la tokenizzazione impiegata (una volta installati, potrò ottenere l'elenco degli LLM disponibili localmente con il comando `list`).

Grazie all'API REST dedicata fornita dagli sviluppatori del progetto, sarà inoltre possibile comunicare con gli LLM installati via codice in modo intuitivo, semplificando la scrittura di applicazioni AI powered.

Se invece l'obiettivo è quello di testare e interagire con il modello in modo diretto, si può ricorrere al comando **ollama run**, che porterà all'esecuzione in locale di un'istanza del LLM desiderato, al quale potremo effettuare query via linea di comando. Saranno infine disponibili anche i comandi **ollama create**, **ollama rm** e **ollama cp**, per rispettivamente: creare un modello da nuovo; eliminare/copiare un'istanza locale (quest'ultima operazione può risultare particolarmente utile in fase di testing o debugging).

Streamlit: introduzione e descrizione del modello di funzionamento

Streamlit è un tool open source (un package Python per l'esattezza, importabile digitando **`import streamlit as st`**) che semplifica e velocizza enormemente la prototipazione e la creazione di applicazioni web interattive basate sui dati [21].

Sfrutta un'architettura di tipo client/server organizzata nel seguente modo: in primo luogo, si avrà un semplice script Python (contenente la logica di business, oltre che da utilizzare come back-end e dunque da server), che verrà eseguito da capo a fondo ad ogni interazione con l'utente (il contesto di esecuzione non andrà però perduto grazie ai meccanismi di mantenimento dello stato della sessione e di caching descritti in seguito); in secondo luogo, uno o più client, che saranno browser web, attraverso i quali l'utilizzatore finale potrà interagire con il prodotto finito [22].



Figura 21 - Logo della libreria Streamlit

Streamlit offre una serie di strumenti integrati per gestire le componenti della nostra applicazione web: in primo luogo, attraverso la funzione **`st.write()`** è possibile renderizzare

all'interno della pagina web generata non solo semplice testo, ma anche il contenuto di variabili, di strutture dati complesse e grafici; in secondo luogo, sono definiti una serie di widget predefiniti (ad esempio **st.slider()**, **st.selectbox()**, **st.button()** e **st.sidebar()** per la gestione del layout generale della pagina), interagendo con cui quali è possibile per l'utente modificare interattivamente il valore di una o più variabili (precedentemente mappate) dello script di back-end; è presente anche un supporto nativo alle applicazioni web con più pagine (si richiamerà il metodo **st.Page()** ogniquale volta sarà necessario definire una nuova pagina, specificando il sorgente associato, il nome da mostrare a schermo e un'icona, e **st.navigation()** per collegarle tra loro); inoltre, per migliorare la reattività e l'esperienza utente, possiamo ricorrere ai decoratori **st.cache_data** (da utilizzare nella stragrande maggioranza dei casi, a meno che non siano coinvolte delle risorse da rendere disponibili globalmente) e **st.cache_resource** (entrambi applicabili a funzioni che restituiscono dei valori che vorremmo salvare nella cache, cosa che risulta particolarmente utile per la comunicazione con i database e il caricamento di LLM); per scrivere invece dei valori all'interno della sessione (cosa che risulterà essere particolarmente utile per mantenere memorizzate delle informazioni ad ogni esecuzione dello script di back-end) si potrà scrivere **st.session_state.key = 'value'** o in alternativa **st.session_state['key'] = 'value'**; sarà infine possibile anche mostrare a schermo immagini conservate sul server Streamlit con il metodo **st.image()**, che avrà come argomento il percorso del file rilevante, e (grazie all'API fornita dagli sviluppatori) definire componenti personalizzati per arricchire il package Streamlit di ulteriori funzionalità [22].

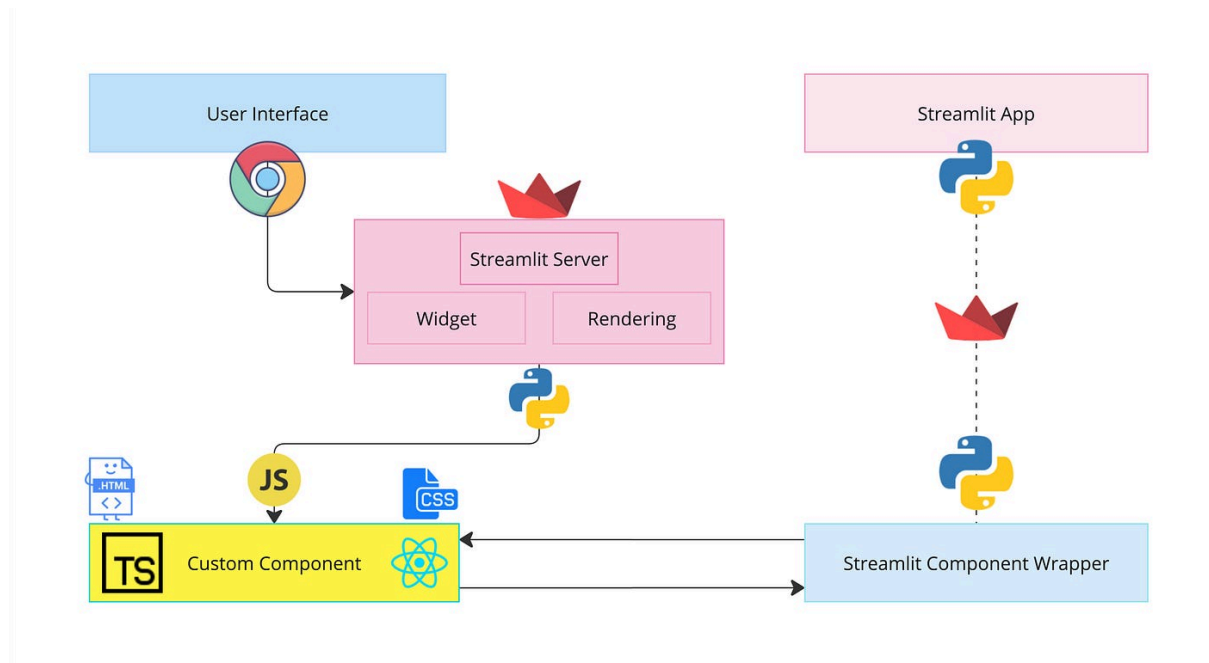


Figura 22 - Schema architetturale dell'utilizzo di Streamlit

Riguardo a quest'ultimo aspetto, si può dire che in generale sono numerose le componenti offerte nativamente nel package Streamlit per costruire svariati tipi di applicazioni web: oltre ai già citati semplici elementi widget, eventualmente anche di form (come `st.number_input()`, `st.text_input()` o `st.text_area()`), abbiamo infatti anche elementi più complessi, come `st.chat_message()` e `st.chat_input()` (per fungere rispettivamente da contenitore di un'interazione testuale passata e da casella di testo per ampliare la conversazione) o anche `st.login()`, `st.logout()` e `st.user` (usati invece per gestire il processo autenticazione degli utenti).

Infine, per occuparsi delle dipendenze dell'intero progetto, sarà sufficiente elencare i pacchetti necessari al funzionamento dello stesso in un file di testo (solitamente chiamato **requirements.txt**) e darlo poi come argomento al gestore di installazione pacchetti nativo di Python, ottenendo il comando: **pip install -r requirements.txt**.

Capitolo III - descrizione e realizzazione del progetto

“Assistente UniBo”

Descrizione del caso d’uso

Come già anticipato nell’introduzione, l’obiettivo di questa tesi è quello di realizzare un chatbot, denominato Assistente UniBo, che sfrutti la metodologia RAG per rispondere a domande riguardanti il regolamento dell’Università di Bologna, consultabile liberamente online [23].

Essendo esso rilasciato sottoforma di poco più di un centinaio di documenti in formato PDF (limitandosi alle sole norme rilevanti per gli studenti), risulta essere un caso di studio ideale per la realizzazione di un sistema RAG di piccole dimensioni impiegante le tecnologie descritte nel capitolo precedente.

Considerazioni iniziali sull’ambiente d’esecuzione

Il server remoto su cui l’applicativo ha avuto modo di eseguire è stato gentilmente fornito dal Centro di Competenza nazionale BI-REX (Big Data Innovation and Research Excellence), con sede a Bologna [24].

L’accesso a tale macchina era garantito dal protocollo ssh (con la possibilità di aprire liberamente le porte dell’elaboratore a scopo di sviluppo) e attraverso l’uso di una VPN.

Le specifiche tecniche più rilevanti sono le seguenti: CPU Intel Xeon con 8 core utilizzabili; 16 GB di RAM; 250 GB di storage; nessuna GPU.

Quest’ultima limitazione è stata particolarmente significativa: le schede grafiche tendono ad essere meglio indicate (sia in termini di consumo energetico che di tempo impiegato nella computazione) in tutte quelle situazioni in cui siano coinvolte grandi moli di dati (come in questo caso), grazie alle loro memorie più performanti e alla loro maggior capacità di parallelizzare (data da un numero di core solitamente nell’ordine delle decine di migliaia) rispetto ai processori standard (più flessibili rispetto alle GPU, ma con un numero di core spesso limitato all’ordine delle decine) [25].

Tenendo conto di ciò, è stato necessario utilizzare una versione quantizzata del LLM scelto, anche per via del quantitativo di RAM relativamente basso: inizialmente si era infatti tentato di usare un modello con quantizzazione a 8 bit, ma la memoria limitata ha reso necessario l’uso della versione con quantizzazione a 4 bit, più leggera (ma anche meno precisa).

La scelta del sistema operativo (Ubuntu 22.04 LTS, invece che il più recente Ubuntu 24.04 LTS) è stata fatta per garantirsi un supporto più esteso e sicuro a quelle che sono le diverse componenti software impiegate per la costruzione del progetto.

```

ubuntu@rag-llm-unibo:~$ neofetch
      .-/+00ssss00+/- .
    `:+ssssssssssssssss+:`
      -+ssssssssssssssssyyssst-
    .osssssssssssssssssdMMMMNyssso.
    /ssssssssshdmmNNmmYNNMMHssssss/
  +ssssssssshmydMMMMMMNdddyssssssst+
  /sssssssshNNMMYhhyyyyhmmNNMMHssssss/
  .ssssssssdMMMNhssssssssshNNMMdssssss.
+ssssshhhyNNMMNyssssssssssyNNMMYssssst+
ossyNNMMNyMMHssssssssssshmmhssssssso
ossyNNMMNyMMHssssssssssshmmhssssssso
+ssssshhhyNNMMNyssssssssssyNNMMYssssst+
.ssssssssdMMMNhssssssssshNNMMdssssss.
  /sssssssshNNMMYhhyyyyhdNNMMHssssss/
  +ssssssssdmydMMMMMMNdddyssssssst+
  /ssssssssshdmmNNmmYNNMMHssssss/
    .osssssssssssssssssdMMMMNyssso.
      -+ssssssssssssssssyyssst-
    `:+ssssssssssssssss+:`
      .-/+00ssss00+/- .

ubuntu@rag-llm-unibo
-----
OS: Ubuntu 22.04.5 LTS x86_64
Host: OpenStack Nova 27.1.1
Kernel: 5.15.0-141-generic
Uptime: 21 days, 1 hour, 55 mins
Packages: 755 (dpkg), 4 (snap)
Shell: bash 5.1.16
Resolution: 1024x768
Terminal: /dev/pts/0
CPU: Intel Xeon Gold 6426Y (8) @ 2.500GHz
GPU: 00:02.0 Red Hat, Inc. Virtio GPU
Memory: 275MiB / 15987MiB

```

Figura 23 - Specifiche della macchina remota utilizzata per il progetto

Considerazioni sulla scelta del LLM e dell'Embedding Model

Tenendo a mente le limitazioni precedentemente elencate, il primo modello che è stato considerato per l'impiego nel chatbot è stata la versione a 7 miliardi di parametri di Llama2 (quella a 13, seppur preferibile, risultava essere troppo pesante in termini di memoria), eseguita in locale grazie alla piattaforma Ollama.

Seppur funzionante, i risultati erano deludenti: il modello sembrava incapace di rispondere con un testo in lingua italiana, cosa che risulta necessario considerando il caso d'uso scelto.

Per tale ragione, la scelta è infine ricaduta su uno dei modelli della famiglia LLaMAntino: **llamantino-2:7b-chat-ultrachat-it-q4_0**, a sua volta una personalizzazione (atta a migliorarne la comprensione e la sintesi di testi in lingua italiana) del modello considerato in precedenza.

Il modello selezionato risulta inoltre (in assenza del contesto fornito dalla documentazione dell'Ateneo), incapace di rispondere ad alcuna domanda riguardante esso, cosa che lo rende dunque particolarmente adatto per questa specifica sperimentazione.

Per quanto riguarda invece la scelta del modello di embedding (anch'esso eseguito in locale), essa è ricaduta su **paraphrase-multilingual-MiniLM-L12-v2** (hostato su Hugging Face) per

via della suo buon rapporto efficienza/prestazioni e del supporto di prima mano alle lingue diverse dall'inglese.

LangChain e LlamaIndex: componenti software utilizzati

Seppur come specificato nel capitolo precedente, le due librerie nascono con scopi e obiettivi diversi, esse sono facilmente integrabili tra loro e forniscono entrambe costrutti per realizzare le parti fondanti di una pipeline RAG.

Nel caso di Assistente UniBo, il grosso di quest'ultima utilizza componenti software forniti da LlamaIndex: SimpleDirectoryReader; VectorStoreIndex; StorageContext; query engine.

Il principale contributo dalla libreria LangChain è dato da un oggetto della classe **ConversationBufferMemory**, usato per il mantenimento dello stato della conversazione.

Architettura logica del progetto

Il chatbot finale avrà dunque un'architettura di tipo client/server (in fase di sviluppo e testing entrambi in esecuzione sullo stesso elaboratore reale): il back-end/server risulterà costituito da una coppia di script python (**web_app.py** e **rag_init.py**) e conterrà tutta la logica di business dell'applicazione (creazione e recupero degli embedding, comunicazione con il LLM ed eccetera); il client web sarà invece generato automaticamente da Streamlit e si occuperà di inoltrare al back-end le richieste effettuate dall'utente, per poi comunicare le risposte sintetizzate dal modello a quest'ultimo.

A questo punto risultano evidenti i vantaggi dell'uso della libreria Streamlit: grazie ad essa, è stato possibile focalizzarsi sullo sviluppo degli aspetti più rilevanti ai fini della tesi, non venendo dunque rallentati da dettagli implementativi di natura secondaria.

Document Processing and Vector Indexing Pipeline

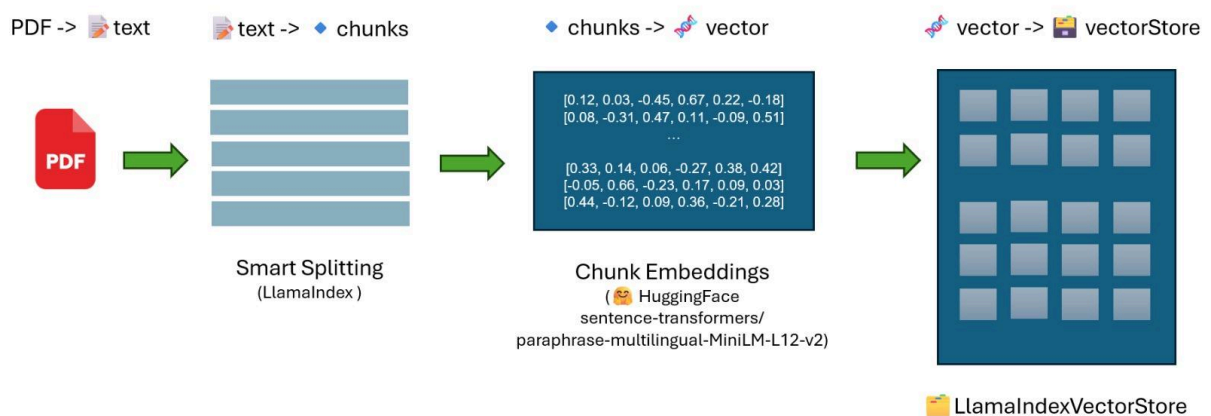


Figura 24 - Schema della pipeline di processing dei documenti e indicizzazione degli embedding

Inoltre, è stato anche realizzato un ulteriore script (**evaluation.py**) con lo scopo di valutare l'effettiva performance dell'intero sistema RAG realizzato, attraverso la generazione di una serie di coppie domanda/risposta e al successivo confronto tra la risposta attesa (quella appartenente al test set) e quella effettivamente generata dal modello in un secondo momento, salvando i risultati ottenuti in forma tabellare all'interno del file **performance.csv**.

Infine, come spiegato nel capitolo precedente, le dipendenze dell'intero progetto verranno gestite attraverso pip e un file **requirements.txt**, che contiene i nomi dei seguenti pacchetti: **llama-index**; **langchain**; **langchain_community**; **ollama**; **pypdf**; **llama-index-llms-ollama**; **llama-index-embeddings-huggingface**; **streamlit**; **pandas**; **faiss-cpu**.

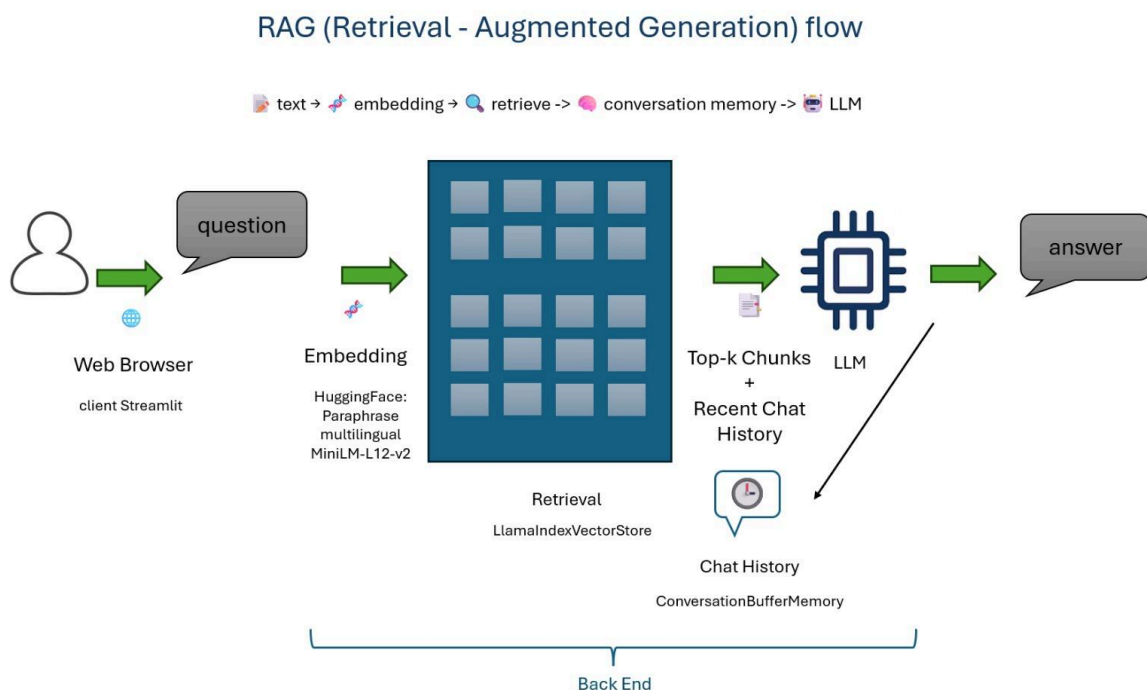


Figura 25 - Schema dell'architettura logica del chatbot realizzato

Diagramma della struttura del progetto (Directory Tree)

```

ubuntu@rag-llm-unibo:~/rag-project$ tree -L 1
.
├── app.py
├── data
├── evaluation.py
├── performance.csv
├── rag_init.py
├── requirements.txt
├── vector_store
├── vector_store.faiss
└── web_app.py
  
```

Figura 26 - Albero dei componenti del progetto

I file citati nella sezione precedente (sorgenti e non) sono collocati nella root della cartella **rag-project**.

Inoltre sono presenti ulteriori sottocartelle (nella figura indicate in blu) per meglio suddividere la struttura del progetto:

- **data**, contenente i documenti scaricati nel sito ufficiale dell'università;
- **vector_store** e **vector_store.faiss**, contenenti rispettivamente gli embedding generati da `rag_init.py` ed `evaluation.py`.

Analisi e commento del sorgente `app.py`

Prima di realizzare `web.py` e `rag_init.py`, si è voluto implementare una versione a linea di comando dell'applicativo chiamata **`app.py`**, allo scopo di testare e sperimentare con le tecnologie in analisi in un contesto più semplice e con meno variabili in gioco.

Si è trattata di una scelta vincente, infatti il codice di `web.py` e `rag_init.py` è molto simile a quello della loro controparte testuale, se non per alcuni accorgimenti che verranno citati e spiegati in seguito.

Va specificato che esso è eseguibile tramite comando: **`python3 app.py`**.

Ora diamo uno sguardo al codice di `app.py`:

```
from llama_index.core import VectorStoreIndex, StorageContext, \
    load_index_from_storage, SimpleDirectoryReader
from llama_index.llms.ollama import Ollama
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from langchain.memory import ConversationBufferMemory
import os, warnings

DOC_DIR = "./data"
VECTOR_STORE_DIR = "./vector_store"
LLM_MODEL = "ifioravanti/llamantino-2:7b-chat-ultrachat-it-q4_0"

llm = Ollama(model=LLM_MODEL, request_timeout=300.0)
embed_model = HuggingFaceEmbedding(
    model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2",
    embed_batch_size=4,
    device="cpu"
)
```

Oltre ad importare i moduli software necessari dai relativi pacchetti e a definire una serie di variabili simboliche relative ai file path rilevanti al progetto, vengono istanziati il LLM e il modello di embedding.

Nel caso del LLM, oltre al nome, viene specificato un **request_timeout** di 300 secondi, che risulta sì piuttosto alto, ma necessario per via delle limitazioni date dall'uso di una CPU per eseguire il modello (come si specifica meglio in seguito, il tempo di risposta può arrivare ad essere anche relativamente alto, a seconda della richiesta effettuata).

Invece, per il modello di embedding, oltre a segnalare il nome dello stesso e il fatto che sia in esecuzione su una CPU, specifico anche una **embed batch_size** di 4, per evitare di saturare la finestra di contesto del LLM.

```
# CONVERSATION MEMORY DEFINITION WITH A FIXED WINDOWS SIZE
memory = ConversationBufferMemory(
    return_messages=True,
    max_token_limit=1000,
    memory_key="chat_history"
)
```

Viene poi istanziato un oggetto `ConversationBufferMemory`, dalla funzione già definita in precedenza.

Significativo è il parametro **max_token_limit** posto a 1000, che limita la lunghezza della conversazione (a cui viene associata come identificativo la **memory_key** "chat_history") per evitare che essa superi in lunghezza la dimensione della context window del modello.

```
# BUILD OR LOAD INDEX
if os.path.exists(VECTOR_STORE_DIR):
    print("Carico l'indice generato in precedenza...")
    storage_context = StorageContext.from_defaults(persist_dir=VECTOR_STORE_DIR)
    index = load_index_from_storage(storage_context, llm=llm, embed_model=embed_model)
else:
    print("Genero un nuovo indice a partire dalle risorse date...")
    documents = SimpleDirectoryReader(DOC_DIR).load_data()
    index = VectorStoreIndex.from_documents(documents, llm=llm, embed_model=embed_model)
    index.storage_context.persist(persist_dir=VECTOR_STORE_DIR)
    print("Indice generato e salvato!")
```

A questo punto viene effettuato un controllo sul `VectorStore`:

- se esso risulta essere pieno, allora viene caricato dello `StorageContext` (nello specifico dal file system) il `VectorStoreIndex` contenente gli embedding generati in precedenza (con un notevole risparmio di tempo);
- viceversa, allora vengono caricati i documenti attraverso un `SimpleDirectoryReader` e generato un `VectorStoreIndex` a partire da essi, che verrà poi salvato in locale attraverso una chiamata al metodo `persist()` dello `StorageContext`.

```
# QUERY LOOP
query_engine = index.as_query_engine(llm=llm)

print("\nPoni un quesito (digita 'chiudi' per uscire)\n")
while True:
    try:
        user_input = input("La tua domanda: ")
        if user_input.lower() in ["esci", "chiudi", "fine"]:
            break
```

```

# PREPARE QUERY WITH CONTEXT
history = memory.load_memory_variables({})["chat_history"]
context = "\n".join([f"{msg.type}: {msg.content}" for msg in history])
augmented_input = f"Conversation Context:\n{context}\n\nCurrent Question: {user_input}"

response = query_engine.query(augmented_input)
response_str = str(response)

# ADD USER INPUT TO CONVERSATION MEMORY
memory.chat_memory.add_user_message(user_input)
# ADD AI RESPONSE TO CONVERSATION MEMORY
memory.chat_memory.add_ai_message(response_str)

print(f"\nRisposta: {response_str}\n")

except Exception as e:
    print(f"\nErrore: {str(e)}")
    continue

```

Lo script termina dunque con il query loop, la parte più significativa dello stesso.

In primo luogo, viene richiesto all’utente di digitare una richiesta testuale via console, salvando il tutto all’interno della variabile **user_input**.

Si verificherà la presenza in tale stringa delle parole chiave “esci”, “chiudi” o “fine” e, in caso affermativo, lo script terminerà.

In caso contrario, verrà salvato il contenuto della memoria precedentemente inizializzata (che sarà una lista di messaggi) all’interno della variabile **history**, per poi concatenarli tutti in un’unica stringa e salvare il risultato nella variabile **context**.

Il modello riceverà in input la stringa **augmented_input**, contenente sia il messaggio corrente che il contenuto di context (cioè i messaggi passati).

Una volta ottenuta una risposta dal modello, essa verrà convertita in stringa e stampata a schermo, dopo aver però prima salvato in memoria la richiesta corrente fatta dall’utente e la relativa risposta generata dal modello.

L’intero processo si ripeterà fino a che l’utente non ne richiederà la fine ricorrendo ad una delle parole chiave.

In caso infine si verifichi un errore, il testo relativo all’eccezione scatenata verrà stampato nella console ed il ciclo riprenderà da capo.

Analisi e commento del sorgente rag_init.py

A questo punto, è lecito chiedersi per quale ragione si è preferito dividere l’applicativo finale in due moduli, cioè:

- rag_init.py, che si occupa dell'inizializzazione delle risorse usate nel back-end, come il caricamento dei due modelli e degli indici vettoriali, oltre che dell'istanziamento del buffer di memoria;
- web_app.py, che gestisce invece la presentazione dell'applicativo, l'interazione con l'utente e i dati memorizzati in sessione.

Fondamentalmente tale scelta è stata fatta per ragioni di leggibilità, modularità e per favorire il processo di testing e debugging in fase di sviluppo.

Analizziamo ora dunque la funzione **initialize_system()** di rag_init.py:

```
# RAG COMPONENTS INITIALIZATION
def initialize_system():
    llm = Ollama(model=LLM_MODEL, request_timeout=300.0)
    embed_model = HuggingFaceEmbedding(
        model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2",
        device="cpu"
    )

    memory = ConversationBufferMemory(
        return_messages=True,
        max_token_limit=1000,
        memory_key="chat_history"
    )

    # BUILD OR LOAD INDEX
    if os.path.exists(VECTOR_STORE_DIR):
        print("Carico l'indice generato in precedenza...")
        storage_context = StorageContext.from_defaults(persist_dir=VECTOR_STORE_DIR)
        index = load_index_from_storage(storage_context, llm=llm, embed_model=embed_model)
    else:
        print("Genero un nuovo indice a partire dalle risorse date...")
        documents = SimpleDirectoryReader(DOC_DIR).load_data()
        index = VectorStoreIndex.from_documents(documents, llm=llm, embed_model=embed_model)
        index.storage_context.persist(persist_dir=VECTOR_STORE_DIR)
        print("Indice generato e salvato!")

    # QUERY LOOP
    query_engine = index.as_query_engine(llm=llm)
    return query_engine, memory
```

Preceduta dagli stessi statement di import presenti in app.py, vediamo che ne riprende gran parte del codice di inizializzazione delle diverse risorse, incapsulandolo in una funzione richiamabile da web_app.py, a cui restituisce il query engine da usare per interrogare il modello e un'istanza della classe ConversationBufferMemory per tenere conto della conversazione.

Analisi e commento del sorgente web_app.py

```
import streamlit as st
from rag_init import initialize_system

# BACKEND INITIALIZATION (from rag_init.py)
query_engine, memory = initialize_system()

st.set_page_config(page_title="Assistente UniBo", page_icon="👤", layout="centered", initial_sidebar_state="expanded")
st.title("👤 Assistente UniBo")
st.caption("Risponde a domande sul regolamento dell'Ateneo!")
st.sidebar.image("https://upload.wikimedia.org/wikipedia/commons/d/d0/Seal_of_the_University_of_Bologna.svg", width=200)
st.sidebar.title("Alma Mater Studiorum - Università di Bologna")

# SESSION STATE INITIALIZATION
if "messages" not in st.session_state:
    st.session_state.messages = [{"role": "assistant", "content": "Ciao! Come posso aiutarti?"}]
    st.session_state.memory = memory

# PAST CONVERSATION HANDLING
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.write(msg["content"])
```

Dopo aver importato i moduli necessari, vengono impostati i seguenti dettagli relativi al front-end web:

- titolo e icona mostrati nella scheda del browser: oltre che il layout della pagina;
- titolo e relativo sottotitolo mostrati a schermo;
- immagine (in questo caso è stato scelto il logo dell'università) e titolo mostrati nella barra laterale.

Successivamente viene inizializzato lo stato della sessione, aggiornandone il valore di due campi:

- **memory** conterrà un riferimento all'oggetto di tipo `ConversationBufferMemory` restituito da `rag_init.py`;
- **messages** verrà utilizzato a sua volta per tenere conto dei messaggi della conversazione avuta tra utente e il LLM, ma a scopo di rendering grafico, invece che per dare stato al modello.

Infine, si esaminerà uno per uno (impiegando un ciclo `for`) gli elementi presenti nel campo `messages` per mostrare a schermo i messaggi della conversazione in corso.

```
# CHAT INPUT HANDLING:
if prompt := st.chat_input("Scrivi la tua domanda..."):
    st.session_state.messages.append({"role": "user", "content": prompt})
```

Viene mostrato a schermo una casella di testo attraverso il quale l'utente può effettuare una richiesta al modello.

Tale stringa, se non nulla, viene salvata nella variabile **prompt** e aggiunta in coda a `messages`.

```
# QUERY HANDLING
with st.spinner("Analizzando la tua domanda..."):
    try:
        # PREPARE QUERY WITH CONTEXT
        history = st.session_state.memory.load_memory_variables({})["chat_history"]
        context = "\n".join([f"{msg.type}: {msg.content}" for msg in history])
        augmented_input = f"Contesto della conversazione:\n{context}\n\nNuova domanda: {prompt}"

        response = query_engine.query(augmented_input)
        response_str = str(response)

        st.session_state.messages.append({"role": "assistant", "content": response_str})
        st.session_state.memory.chat_memory.add_user_message(prompt)
        st.session_state.memory.chat_memory.add_ai_message(response_str)

        # FORCE UI UPDATE
        st.rerun()

    except Exception as e:
        st.error(f"Errore del sistema: {str(e)}")
        st.session_state.messages.append({"role": "assistant", "content": "Mi dispiace, c'è stato un errore."})
```

Il query loop risulta piuttosto simile a quello precedentemente descritto per app.py con la differenza che oltre al valore di memory (a cui vengono aggiunte domanda e risposta), anche quello del campo messages viene analogamente aggiornato con l'output del modello in caso di successo e con un messaggio di errore in caso di problemi.

Un altro dettaglio da notare è che a fine ciclo viene richiesto l'aggiornamento forzato del front-end grafico, in modo che venga effettuato il rendering a schermo dell'intera conversazione (compresi i messaggi più recenti).

Inoltre è importante notare come si ricorra all'operatore **with** per renderizzare un'animazione di attesa fino al completamento della sintesi della risposta da parte del LLM.

In conclusione, va notato che tale script risulta eseguibile tramite comando:

streamlit web_app.py.

Analisi e commento del sorgente evaluation.py - introduzione

A differenza degli altri script, che impiegavano principalmente costrutti forniti dalla libreria LlamaIndex e che usavano LangChain solo marginalmente, è stato deciso di usare solamente quest'ultima per la realizzazione di evaluation.py, in modo da sviluppare una conoscenza di base adeguata (a livello pratico) anche di quest'ultima.

Il comando per eseguirlo sarà però invece molto simile ai precedenti:

python3 evaluation.py.

Analizziamone dunque il codice:

```

import os
from langchain_core.output_parsers import StrOutputParser
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.llms import Ollama
from langchain_core.prompts import ChatPromptTemplate
from langchain.chains.retrieval import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
import pandas as pd
import re
from typing import List, Dict

DOC_DIR = "./data"
LLM_MODEL = "ifioravanti/llamantino-2:7b-chat-ultrachat-it-q4_0"
EMBEDDING_MODEL = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"

```

In primo luogo, si procede ad importare i moduli necessari per il funzionamento dello script e successivamente a definire, come in precedenza, variabili simboliche per i file path rilevanti. Particolarmente significativi sono i pacchetti pandas e re (il cui ruolo nello script verrà reso chiaro in seguito), necessari rispettivamente: per la manipolazione di dati strutturati e per la gestione delle regex.

```

llm = Ollama(
    model=LLM_MODEL,
    base_url="http://localhost:11434",
    timeout=300,
    temperature=0.3,
    num_ctx=2048
)

embed_model = HuggingFaceEmbeddings(
    model_name=EMBEDDING_MODEL,
    model_kwargs={"device": "cpu"},
    encode_kwargs={"normalize_embeddings": True}
)

```

Vengono dunque istanziati i due modelli, seguendo la sintassi di LangChain.

Particolarmente rilevanti sono i parametri di inizializzazione del LLM, che ci permettono di specificare:

- il nome del modello (**model**);
- l'URL e la porta associati (**base_url**);
- la temperatura (**temperature**), in questo caso posta a 0.3 per garantire risposte precise;

- la dimensione della finestra di contesto (**num_ctx**), in questo caso posta a 2048 per evitare di saturare la memoria volatile.

Analisi e commento del sorgente `evaluation.py` - funzione `load_documents()`

```
def load_documents():
    loader = DirectoryLoader(
        DOC_DIR,
        glob="**/*.pdf",
        loader_cls=PyPDFLoader,
        show_progress=True
    )
    docs = loader.load()

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=500,
        chunk_overlap=100
    )
    return text_splitter.split_documents(docs)
```

Si definisce dunque la funzione **load_documents()**, responsabile della divisione dei documenti di testo originali in chunk, che vengono poi restituiti al chiamante.

Per fare ciò si istanzia una classe **DirectoryLoader**, specificando i seguenti parametri:

- **DOC_DIR**, cioè il percorso completo della cartella **data**;
- **glob** settato a `"**/*.pdf"`, che porterà il loader a considerare tutti i file in formato PDF presenti (anche in eventuali sottocartelle);
- **loader_cls** con valore **PyPDFLoader**, coerente con la tipologia di documenti considerata;
- **show_progress** impostato a `true`, in modo che lo stato del processo sia segnalato a schermo da una barra di caricamento.

Dopo aver effettivamente caricato il contenuto dei documenti in memoria (grazie al metodo **load()** della classe loader), un oggetto della classe **RecursiveCharacterTextSplitter** si occuperà della generazione dei chunk (attraverso l'invocazione del metodo **split_documents()**).

A tal proposito vengono specificate anche la dimensione dei chunk espressa in caratteri (tramite il parametro **chunk_size**) e allo stesso modo anche la dimensione dell'overlap (numero di caratteri condivisi tra chunk successivi), cosa che risulta necessario per mantenere la coerenza semantica in caso di testo diviso.

Analisi e commento del sorgente `evaluation.py` - funzione `generate_test_set()`

```
# TEST Q/A GENERATION (+ CALL TO FALLBACK FUNCTION FOR ERROR HANDLING)
def generate_test_set(split_docs, num_questions=10):
    try:
        sample_docs = split_docs[:3]
        context_text = "\n\n".join([doc.page_content[:500] for doc in sample_docs])

        # FORMAT USED TO GENERATE TEST Q/A
        prompt_template = """
        DEVI generare ESATTAMENTE {num_questions} coppie di domande e risposte.
        USA SOLO QUESTO FORMATO SENZA ECCEZIONI:

        INIZIO ESEMPIO
        1. Q: [Testo della domanda specifica sul contesto]
        1. A: [Testo della risposta precisa dal contesto]
        2. Q: [Seconda domanda specifica]
        2. A: [Seconda risposta precisa]
        FINE ESEMPIO

        CONTESTO:
        {context}

        REGOLE:
        - Numerare ogni coppia (1., 2., ...)
        - Usare "Q:" e "A:" per ogni domanda/risposta
        - Le domande devono essere SPECIFICHE al contesto
        - NESSUN altro testo oltre alle domande/risposte numerate
        """
```

Successivamente si ha la funzione `generate_test_set()` con argomenti il vettore contenente i chunk generati e il numero massimo di coppie domanda/risposta da generare.

In primo luogo, per non saturare la context window, si consideravano solamente i primi tre chunk di testo ottenuti come input e li si salva nella variabile `sample_docs`.

Iterando su di essi e aggregandone il contenuto insieme, verrà generata la stringa `context_text`.

La stringa `prompt_template` contiene invece una descrizione ad alto livello (in cui i valori di `context` e `num_question` sono parametrizzati) del formato richiesto al modello per la generazione del test set.

```
prompt = ChatPromptTemplate.from_template(prompt_template)
chain = prompt | llm | StrOutputParser()

result = chain.invoke({
    "context": context_text,
    "num_questions": num_questions
})
```

Viene dunque creato un oggetto prompt dinamico (che permette cioè al programmatore di variare liberamente i valori di `context` e `num_question`) usando il metodo `from_template()` della classe **ChatPromptTemplate** (richiesto da `LangChain`, ma non da `LlamaIndex`, per cui è richiesto di usare una semplice stringa).

Una volta fatto ciò, si crea dunque una catena di passaggi da eseguire in sequenza e con input/output rediretti (rappresentata dall'oggetto **chain**): il prompt viene dunque dato in input al modello la cui risposta viene tradotta in stringa da un oggetto di tipo **StrOutputParser**.

Otterremo il risultato di tale catena invocando su di essa il metodo **invoke()** e fornendole i parametri richiesti.

```
if not all(x in result for x in ["Q:", "A:", "1.", "2."]):
    raise ValueError("LLM deviated from required format")

return parse_qa(result)

except Exception as e:
    print(f"Question generation failed: {str(e)}")
    print("Using fallback questions...")
    return create_fallback_questions(sample_docs)
```

Infine, viene controllato il formato della risposta generata:

- se esso è corretto, la variabile **result** viene restituita al chiamante;
- viceversa, al chiamante viene restituito il risultato della funzione **create_fallback_questions()**, che viene dunque invocata.

Analisi e commento del sorgente evaluation.py - funzione create_fallback_questions()

```
def create_fallback_questions(docs):
    questions = []

    for i, doc in enumerate(docs[:5]): # Limit to 5 docmunets to get 10 questions
        content = doc.page_content[:500] # First 500 chars

        questions.extend([
            {
                "question": f"Qual è il contenuto principale del documento {i+1}?",
                "answer": f"Il documento contiene informazioni su: {content[:150]}..."
            },
            {
                "question": f"Quali sono i dettagli specifici discussi nel documento {i+1}?",
                "answer": f"Nel documento si discute dettagliatamente di: {content[100:250]}..."
            }
        ])

    print(f"Created {len(questions)} fallback questions")
    return questions
```

Com'è possibile notare, create_fallback_questions() ha un comportamento abbastanza simile a quello della funzione precedente, se non per quanto riguarda l'effettiva generazione delle coppie domanda/risposta: invece di farle generare dal LLM, esse riportano integralmente il contenuto dei chunk nel caso delle risposte e sono invece non significative nel caso delle domande.

Ciò le rende chiaramente più semplici da realizzare (utile in fase di realizzazione o debug), ma anche piuttosto limitate e restrittive ai fini della tesi.

Analisi e commento del sorgente evaluation.py - funzione parse_qa()

```
def parse_qa(text):
    qa_pairs = []
    current_pair = {"question": "", "answer": ""}

    # NORMALIZE TEXT FORMAT
    text = text.replace("?", "? ")
    lines = [line.strip() for line in text.split("\n") if line.strip()]

    # SUPPORTED PATTERNS
    number_pattern = r"^\d+\.\?\s*"
    q_patterns = [r"Q[:.]", r"Domanda[:.]", r"\? $" ]
    a_patterns = [r"A[:.]", r"Risposta[:.]" ]

    for line in lines:
        try:
            # REMOVE NUMBERING
            clean_line = re.sub(number_pattern, "", line)

            # CHECK PRESENCE OF QUESTIONS
            if any(re.search(pattern, clean_line, re.IGNORECASE) for pattern in q_patterns):
                if current_pair["question"] and current_pair["answer"]:
                    qa_pairs.append(current_pair)
                    current_pair = {"question": "", "answer": ""}

                # EXTRACT QUESTION TEXT
                question_text = clean_line.split(":", 1)[1] if ":" in clean_line else clean_line
                current_pair["question"] = question_text.strip()

        except Exception as e:
            print(f"Warning: Error parsing line - {str(e)}")
            continue

    if current_pair["question"] and current_pair["answer"]:
        qa_pairs.append(current_pair)

    if not qa_pairs:
        print(f"No Q&A pairs found in text:\n{text[:200]}...")
    else:
        print(f"Successfully parsed {len(qa_pairs)} Q&A pairs")

    return qa_pairs
```

Analizziamo ora la funzione **parse_qa()**, che riceve in input il testo generato da **generate_test_set()** o **create_fallback_questions()**, a seconda dei casi.

Vengono poi definiti: la lista **qa_pairs**, che conterrà l'elenco delle coppie domanda/risposta e il dizionario **current_pair**, necessario per la costruzione della coppia.

Il testo viene poi normalizzato e salvato nella variabile **lines**:

- viene aggiunto uno spazio dopo il punto interrogativo, in modo da ben separare domanda e risposta (invocando **replace("?", "? ")** su **text**);
- il testo viene diviso in righe (grazie al metodo **split("\n")**), ciò significa che **lines** sarà dunque una lista;

- vengono rimossi gli spazi bianchi ed escluse le linee vuote con `strip()`.

Si definiscono quindi le regex per riconoscere: le numerazioni di inizio riga (tramite stringa **number_pattern**) e domande e risposte (variabili **q_patterns** e **a_patterns**).

Si itera dunque su `lines` tramite un ciclo `for` e per ogni riga (o **line**):

- viene rimossa la numerazione (invocando il metodo **sub()** con argomento `number_pattern`);
- se è una domanda (controllo effettuato dal metodo **search()**) viene aggiunta in `current_pair` come tale;
- se è una risposta (controllo effettuato dal metodo **search()**) viene aggiunta in `current_pair` nel campo `answer`;
- se `current_pair` contiene una coppia domanda/risposta valida, il suo contenuto viene aggiunto in coda a `qa_pairs`;
- in caso di eccezioni, viene segnalato con un messaggio di errore a schermo, ma poi il ciclo continua.

Se tutto è andato bene, viene infine stampato a schermo il numero di coppie generate; in caso invece non ne sia stata generata neanche una, sarà notificato un messaggio di errore.

Analisi e commento del sorgente evaluation.py - classe LocalEvaluator

```
class LocalEvaluator:
    def __init__(self, llm):
        self.llm = llm

    def evaluate(self, prediction, reference, question):
        """Evaluate prediction against reference for given question."""
        prompt_text = f"""
        Valuta la qualità di questa risposta da 0 a 100:

        Domanda: {question}
        Risposta di riferimento: {reference}
        Risposta da valutare: {prediction}

        Fornisci solo un punteggio numerico tra 0 e 100.
        """

        try:
            response = self.llm.invoke(prompt_text)
            # NUMERIC SCORE EXTRACTION
            import re
            numbers = re.findall(r'\d+', str(response))
            if numbers:
                score = int(numbers[0])
                return {"score": score / 100}
            else:
                return {"score": 0}
        except Exception as e:
            print(f"Error in evaluation: {str(e)}")
            return {"score": 0}
```

Questa è invece la definizione della classe **LocalEvaluator**, caratterizzata da due metodi: **__init__()**, cioè il suo costruttore (piuttosto banale, essendo che aggiorna semplicemente il suo riferimento locale al LLM) e **evaluate()**, che, con un procedimento analogo a quello che è stato spiegato in precedenza nel caso di **generate_test_set()**, usa un prompt dinamico strutturato per richiedere al modello di dare un voto da 0 a 100 alla somiglianza tra la risposta di riferimento (che fa parte del test set) e quella da valutare (generata in un secondo momento dal modello).

Infine, estrae dalla stringa data in output dal LLM tutti i numeri trovati (usando un regex e invocando il metodo **findall()**), per poi fare un cast ad intero e restituire al chiamante il valore ottenuto.

Se non vengono trovati numeri o si è verificato un qualche errore, viene restituito 0.

Analisi e commento del sorgente `evaluation.py` - funzione `evaluate_rag()`

```
def evaluate_rag(test_set, vector_store):
    if not test_set:
        print("Error: No test questions available for evaluation")
        return pd.DataFrame()

    evaluator = LocalEvaluator(llm)
    results = []

    print(f"Evaluating {len(test_set)} questions...")

    for i, qa in enumerate(test_set):
        print(f"Processing question {i+1}/{len(test_set)}")

        try:
            # RELEVANT DOCS RETREIVAL
            docs = vector_store.similarity_search(qa["question"], k=4)
            context = "\n\n".join([d.page_content for d in docs])

            prompt_text = f"""
            Rispondi a questa domanda basandoti solo sul contesto dato:
            Domanda: {qa["question"]}

            Contesto:
            {context}

            Rispondi brevemente:
            """
```

Ecco dunque la funzione che farà uso della classe appena descritta: **`evaluate_rag()`**, che avrà come argomenti il test set generato e il **VectorStore** e che restituirà un **DataFrame**, una struttura dati organizzata in righe e colonne (fondamentalmente una tabella).

Ogni riga risulta identificata da un indice, mentre le colonne (che potranno contenere tipi diversi di dati), da un nome.

Dopo aver controllato che il test set non sia vuoto (in quel caso viene restituito un **DataFrame** vuoto a sua volta) e aver creato un'istanza di **LocalEvaluator**, si procede a, per ogni coppia domanda/risposta:

- selezionare i 4 documenti più rilevanti alla domanda considerata (invocando il metodo **`similarity_search()`** del **Vector Store**);
- salvare nella variabile **context** il contenuto aggregato di tali documenti;
- invocare il **LLM**, fornendogli il contesto appena selezionato;

- ottenere un punteggio numerico invocando il metodo `evaluate()` di `LocalEvaluator`;
- aggiungere in coda alla lista di **results** (definita all'inizio) una struttura dizionario con i campi **question**, **source**, **prediction**, **reference** e **sources** (riempiti con i valori relativi all'iterazione corrente).

In caso di errore invece, in coda alla lista di **results** (definita all'inizio) verrà posta una struttura dizionario con gli stessi campi della precedente, ma settati a valori nulli.

```
prediction = llm.invoke(prompt_text)

# PREDICTION EVALUATION
score = evaluator.evaluate(
    prediction=prediction,
    reference=qa["answer"],
    question=qa["question"]
)

results.append({
    "question": qa["question"],
    "score": score["score"],
    "prediction": prediction,
    "reference": qa["answer"],
    "sources": [d.metadata.get("source", "unknown") for d in docs]
})

except Exception as e:
    print(f"Error processing question {i+1}: {str(e)}")
    results.append({
        "question": qa["question"],
        "score": 0.0,
        "prediction": f"Error: {str(e)}",
        "reference": qa["answer"],
        "sources": []
    })

return pd.DataFrame(results)
```

Infine, verrà restituito dalla funzione un `DataFrame` creato a partire dai valori presenti in `result`.

Analisi e commento del sorgente evaluation.py - funzione visualize_performance()

```
def visualize_performance(results_df):
    try:
        # CONVERT TO DATAFRAME, IF NECESSARY (ERROR HANDLING)
        if not isinstance(results_df, pd.DataFrame):
            results_df = pd.DataFrame(results_df)

        # CHECK FOR REQUIRED COLUMNS (ERROR HANDLING)
        if 'question' not in results_df.columns or 'score' not in results_df.columns:
            print("Error: Missing required columns (question, score)")
            return

        # CLEAN AND PREPARE DATA (ERROR HANDLING)
        results_df = results_df.copy()
        results_df['score'] = pd.to_numeric(results_df['score'], errors='coerce')
        results_df = results_df.dropna(subset=['score'])

        # BUILD EMPTY TABLE
        display_df = pd.DataFrame({
            'Domanda #': [f"Q{i+1}" for i in range(len(results_df))],
            'Domanda': results_df['question'].str[:80] + '...', # Truncate long questions
            'Punteggio': results_df['score'].round(3),
            'Giudizio': ['✓ Buono' if score >= 0.7 else '△ Medio' if score >= 0.4 else 'X Scarso'
                        for score in results_df['score']]
        })
    }
```

Terminando l'analisi del codice delle funzioni di evaluation.py, ne è presente anche per la stampa a console del DataFrame generato: **visualize_performance()**.

Dopo aver fatto una serie di controlli sulla validità del DataFrame ricevuto in input, viene costruito un secondo DataFrame più adatto alla visualizzazione:

- le domande troppo lunghe vengono dunque troncate;
- i punteggi arrotondati;
- i giudizi espressi tramite caratteri visivi.

```
# PRINT THE DISPLAY TABLE
print("\n" + "="*100)
print("RISULTATI DI VALUTAZIONE PERFORMANCE DEL SISTEMA RAG")
print("="*100)
print(display_df.to_string(index=False, max_colwidth=80))

# PRINT SUMMARY STATISTICS
avg_score = results_df['score'].mean()
print("\n" + "-"*50)
print("STATISTICHE RIASSUNTIVE: ")
print("-"*50)
print(f"Numero domande: {len(results_df)}")
print(f"Punteggio medio: {avg_score:.3f}")
print(f"Punteggio massimo: {results_df['score'].max():.3f}")
print(f"Punteggio peggiore: {results_df['score'].min():.3f}")
print("="*100)

except Exception as e:
    print(f"Error displaying results: {str(e)}")
    print("\nRaw data preview:")
    print(results_df.head() if isinstance(results_df, pd.DataFrame) else str(results_df)[:500])
```

Vengono dunque stampati sulla console il DataFrame e una serie di statistiche riassuntive. Inoltre, come nei casi precedenti, in caso di errori essi sono segnalati a schermo.

Analisi e commento del sorgente `evaluation.py - main()`

```
def main():
    try:
        if not os.path.exists(DOC_DIR):
            print(f"Error: Directory {DOC_DIR} not found. Please create it and add PDF files.")
            return

        print("Caricando e processando i documenti...")
        split_docs = load_documents()

        if not split_docs:
            print("Error: No documents found. Please add PDF files to the data directory.")
            return

        print(f"Caricati {len(split_docs)} chunk di documenti")

        print("Creando il vector store...")
        vector_store = FAISS.from_documents(split_docs, embed_model)

        print("Generando le domande di test...")
        test_set = generate_test_set(split_docs)

        if not test_set:
            print("Error: Could not generate any test questions. Check your LLM connection.")
            return

        print(f"Sono state generate {len(test_set)} domande di test")
```

In conclusione, bisogna dunque soffermarsi sul codice della funzione `main()`.

Dopo aver controllato l'esistenza della cartella **data**, la variabile **split_docs** viene riempita con la lista di chunk restituita dal metodo `load_documents()`.

Quindi, in seguito ad un controllo sulla validità di `split_docs`, viene generato dapprima il Vector Store e poi il test set (se esso risulta vuoto, il programma termina), in entrambi i casi a partire da essa.

Ognuno di tali passaggi viene inoltre segnalato all'utente tramite console attraverso l'uso di funzioni di **print()**.

```

print("\Domande di test:")
for i, qa in enumerate(test_set[:3]):
    print(f"Q{i+1}: {qa['question']}")
    print(f"A{i+1}: {qa['answer'][:100]}...")
    print()

print("Valutando il sistema RAG...")
results = evaluate_rag(test_set, vector_store)

if results.empty():
    print("Error: No evaluation results generated.")
    return

results.to_csv("performance.csv", index=False)
print("Risultati salvati in performance.csv")

print("Visualizzando i risultati...")
visualize_performance(results)

except Exception as e:
    print(f"Error in main execution: {str(e)}")
    import traceback
    traceback.print_exc()
    raise

if __name__ == "__main__":
    main()

```

Analogamente, dopo aver stampato le coppie domanda/risposta generate, viene invocata la funzione `evaluate_rag()` ed il suo valore di ritorno viene salvato nella variabile **results**.

Dopo aver controllato che quest'ultima non sia vuota, i risultati vengono salvati nella memoria di massa attraverso il metodo `to_csv()` e successivamente mostrati sullo schermo grazie alla funzione `visualize_performance()`.

Infine, in caso di eccezioni e come in precedenza, viene stampato a schermo un messaggio d'errore.

Question	Prediction	Reference	Sources	Score
Come funziona il tutorato per l'Unibo?	Il tutorato all'Università di Bologna è un sistema di supporto personalizzato che offre agli studenti la possibilità di formare una relazione con i tutor (...).	Gli assegnisti di tutorato svolgono una funzione di interfaccia tra lo studente e la struttura universitaria (...).	Reg.Assegni di Tutorato.pdf	75

Figura 27 - Esempio di tupla di `performance.csv`

Considerazioni sulle performance generali del chatbot

Come già specificato in precedenza, la configurazione hardware della macchina remota su cui l'intero applicativo viene eseguito ha fortemente influenzato la scelta delle sue componenti software e lo sviluppo dello stesso.

A tal proposito, le performance finali del chatbot non sono da meno: l'assenza di una GPU (e la conseguente necessità di eseguire il LLM sulla CPU del calcolatore) ha di molto dilatato i tempi di generazione di una risposta, che solitamente si aggirano tra i 2 e 3 minuti (supponendo di avere un singolo cliente collegato), a seconda dei casi.

Questo risultato non è stato però una sorpresa: è infatti noto che le GPU di fascia alta sono generalmente in grado di produrre (soprattutto quando si parla di modelli di grandi dimensioni) una quantità di token al secondo fino a venti volte superiore a quella di una CPU di analoga categoria [26], oltre ad essere più efficienti da un punto energetico e dunque ambientale [27].

Per quanto riguarda invece la valutazione della scalabilità all'aumentare del traffico delle richieste e del numero degli utenti da servire:

- dal punto di vista del front-end Streamlit, la necessità di eseguire da capo il sorgente Python sottostante ad ogni interazione è un collo di bottiglia abbastanza considerevole, al punto che, se tale aspetto fosse una priorità, potrebbe valere la pena spostare gran parte delle operazioni computazionalmente intensive a processi separati per migliorare l'usabilità generale [28];
- dal punto di vista invece del back-end e dalla gestione dell'istanza locale del LLM, non è realistico pensare di servire più di due o tre utenti per via delle limitazioni hardware imposte dal sistema.

In linea generale, per permettere all'applicazione di scalare significativamente, sarebbe necessario adottare un'architettura basata su containerizzazione (ad esempio tramite Docker), combinata ad un'integrazione orizzontale su più nodi, così da garantire una maggiore resilienza e capacità di carico all'intero sistema [29].

Assistente UniBo: galleria di screenshot

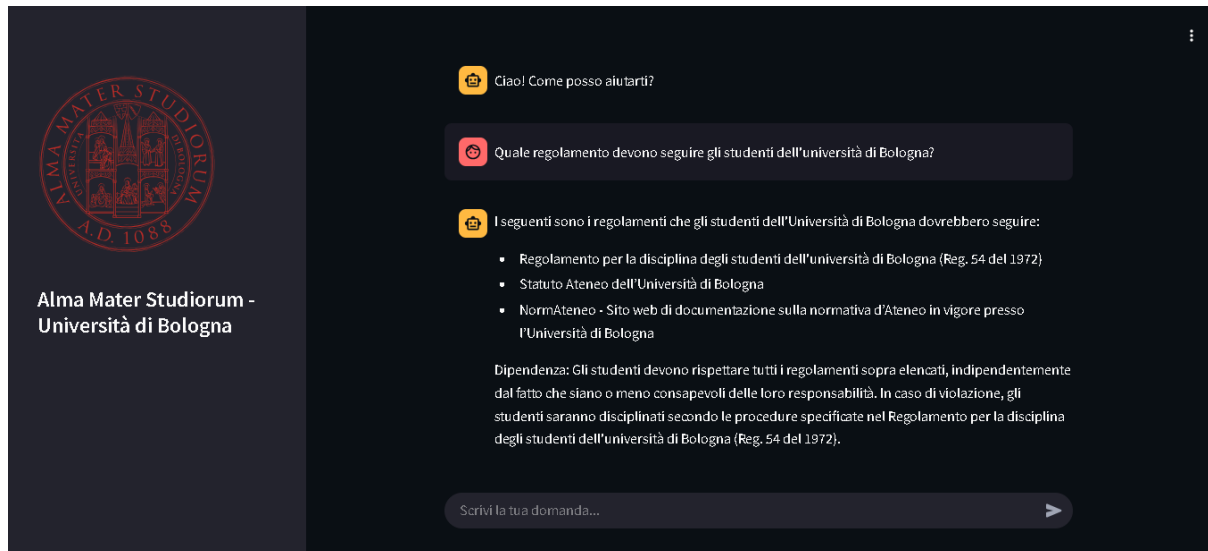


Figura 28 - Schermata 1 di interazione con il chatbot

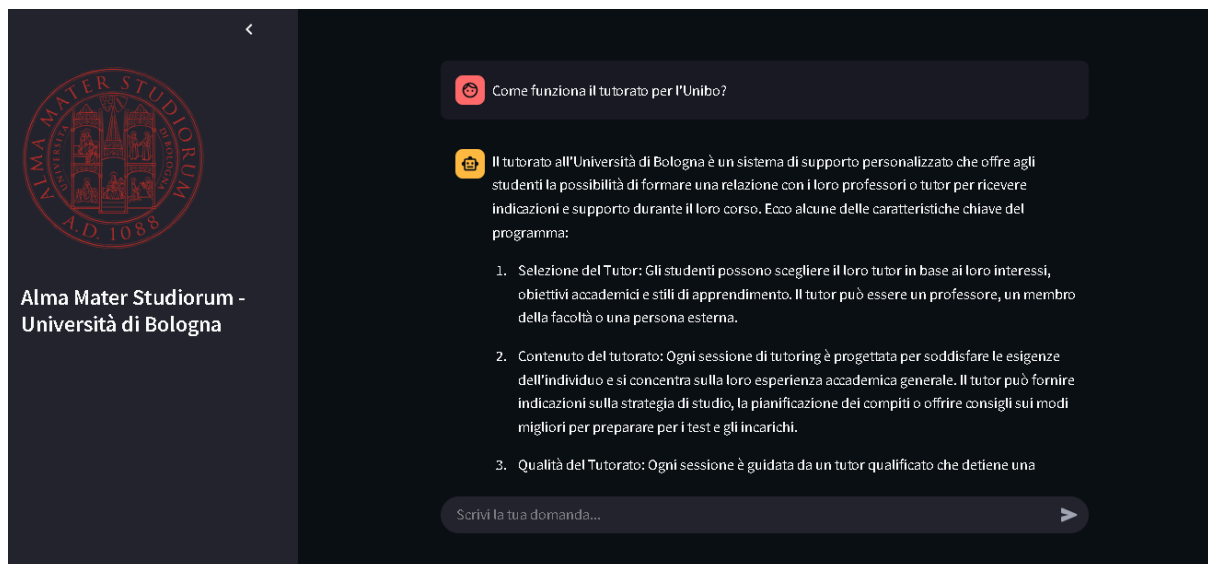


Figura 29 - Schermata 2 di interazione con il chatbot

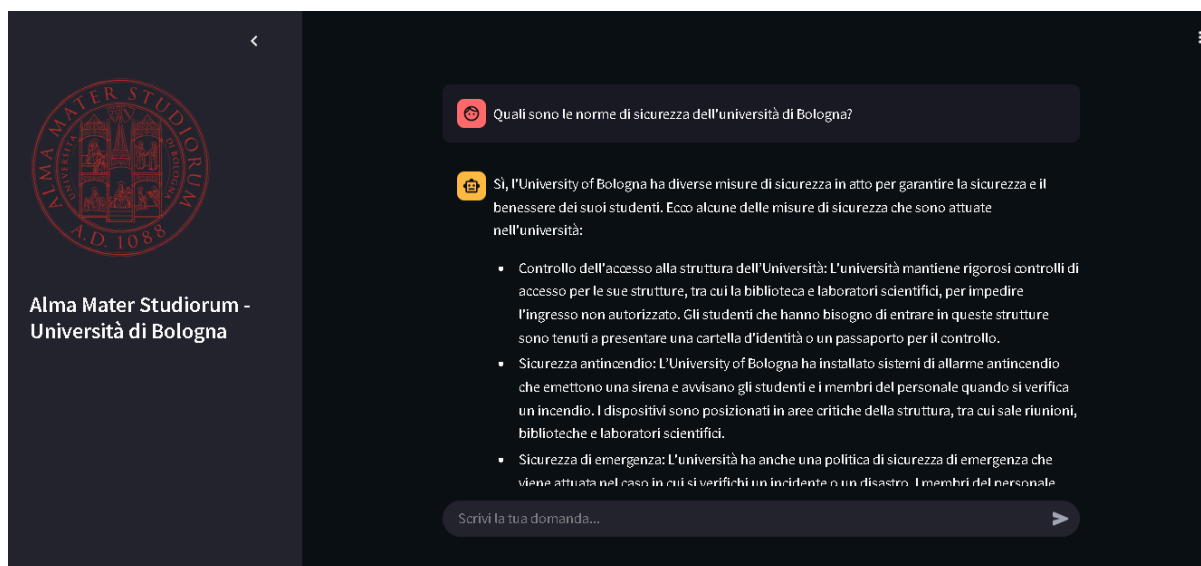


Figura 30 - Schermata 3 di interazione con il chatbot

Conclusioni

Il lavoro svolto ha dunque dimostrato le enormi potenzialità delle tecnologie impiegate, anche per un caso d'uso ristretto come quello considerato: non solo per quanto riguarda gli LLM (e le intelligenze artificiali generative in senso lato), ma anche per la metodologia RAG e le tecniche di retrieval.

Il sistema realizzato si è infatti dimostrato in grado di comprendere e rispondere efficacemente alle domande postegli sugli argomenti trattati nel corpus di documenti considerato, pur con vincoli fisiologici dati dalla semplicità della sua architettura logica e dalle limitazioni hardware, in assenza delle quali si sarebbero potuti utilizzare dei modelli ben più performanti e sarebbe stato anche possibile utilizzare una metodologia di testing più rigorosa e precisa (che impieghi magari un secondo LLM per la validazione o per la generazione delle domande di test, oltre che l'impiego di uno o più degli strumenti di validazione avanzata offerti da LlamaIndex descritti in precedenza).

Detto questo, sono diversi in realtà i possibili sviluppi futuri che si potrebbero considerare anche tenendo conto di tali limitazioni:

- in primo luogo, sarebbe interessante implementare una qualche sorta di sistema di caching locale (sia in memoria centrale che eventualmente in secondaria), in modo da diminuire drasticamente i tempi di attesa di una risposta da parte del modello (di per sé piuttosto lunghi in questi momento);
- in secondo luogo, varrebbe anche la pena introdurre la capacità per le terze parti di interrogare l'intero sistema via codice tramite API Rest (definite in standard OpenAPI), mantenendo il modulo **init_rag.py** così com'è e scrivendo un ulteriore modulo che sia costruito sopra di esso e che possa offrire i servizi in questione (proprio com'è stato fatto con **web_app.py**);
- infine, varrebbe la pena di espandere il corpus di documenti utilizzato nel sistema, includendo in esso ulteriori testi rilevanti all'ambito universitario, in modo da estenderne il contesto di utilizzo, o aggiungendo materiale in lingua straniera per un eventuale supporto multilingua.

Bibliografia

1. <https://unitesi.unive.it/retrieve/e2acf372-55b7-4515-a5ac-8814c4c9f9f4/866665-1269806.pdf>
2. <https://www.ibm.com/think/topics/large-language-models>
3. <https://aws.amazon.com/it/what-is/foundation-models/>
4. <https://arxiv.org/pdf/2005.11401>
5. <https://dev.to/codemaker2015/the-ultimate-guide-to-retrieval-augmented-generation-rag-5e6e>
6. <https://www.datacamp.com/blog/rag-advanced>
7. <https://www.linkedin.com/pulse/retrieval-techniques-sparse-dense-hybrid-najeeb-khan-ph-d--wmtpc/>
8. <https://www.newscatcherapi.com/blog/ultimate-guide-to-text-similarity-with-python>
9. <https://www.linkedin.com/pulse/embedding-similarity-measures-sangeetha-venkatesan/>
10. <https://www.hopsworx.ai/dictionary/retrieval-augmented-generation-llm>
11. <https://python.langchain.com/docs/>, con sezioni specifiche riguardanti: le interfacce dei ChatModel, i tool, il caching, la memoria e gli agenti
12. <https://www.ibm.com/it-it/think/topics/llamaindex-vs-langchain>
13. https://github.com/run-llama/llama_index
14. <https://docs.llamaindex.ai/en/stable/>, con sezioni specifiche riguardanti: i connettori di dati, gli indici di dati, i modelli di embedding, query/chat engine, workflow ed evaluation
15. <https://www.ibm.com/it-it/think/topics/llama-2>
16. <https://www.ibm.com/it-it/think/topics/quantization>
17. <https://huggingface.co/swap-uniba>
18. <https://github.com/marcopoli/LLaMAntino-2-ITA>
19. <https://www.cohorte.co/blog/a-comprehensive-guide-to-ollama>

20. <https://www.geeksforgeeks.org/artificial-intelligence/ollama-explained-transforming-a-i-accessibility-and-language-processing/>
21. <https://github.com/streamlit/streamlit>
22. <https://docs.streamlit.io/>
23. <https://normateneo.unibo.it/regolamenti>
24. <https://bi-rex.it/chi-siamo/>
25. https://medium.com/@mysterious_obscure/computation-of-llms-cpu-vs-gpu-detailed-analysis-5e04a3957d7e
26. <https://medium.com/@mental-complex.ai/local-llms-how-well-do-cpus-and-gpus-really-perform-a-practical-ai-benchmark-6793fc683f13>
27. <https://massedcompute.com/faq-answers/?question=What+are+the+power+consumption+differences+between+NVIDIA+GPUs+and+CPUs+for+large+language+model+training%3F>
28. <https://discuss.streamlit.io/t/make-apps-faster-by-moving-heavy-computation-to-a-separate-process/68541>
29. <https://ploomber.io/blog/scaling-streamlit/>
30. https://github.com/Tomassone/Assistente_UniBo (collegamento al repository GitHub pubblico del progetto)